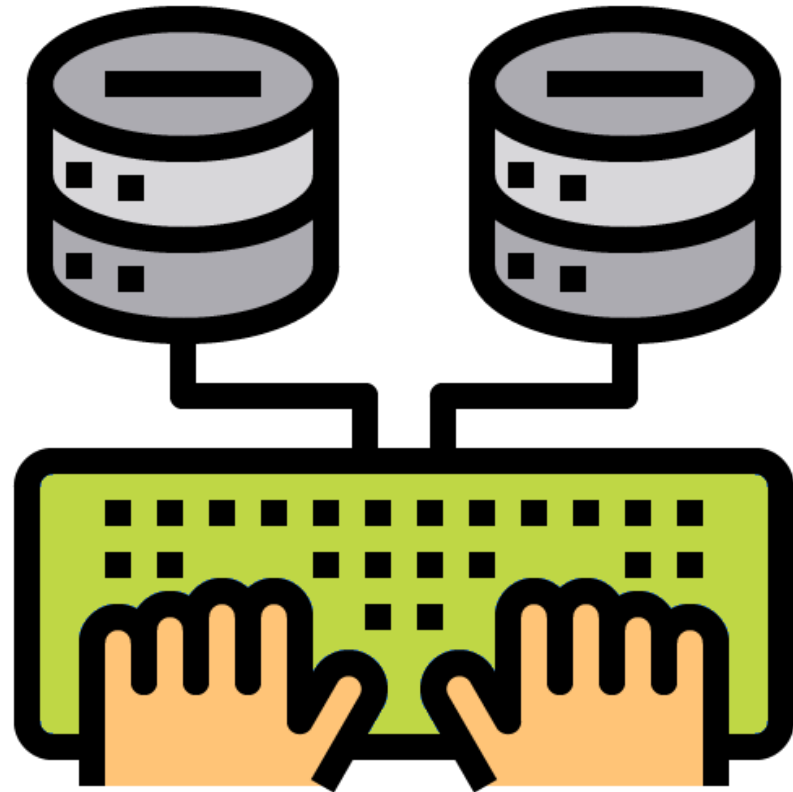




Big Data Analytics

Big data platforms III MapReduce, YARN, SPARK

Lecturer:

Márcia Lourenço Baptista

m.baptista@novaims.unl.pt

Big Data Analytics Platform

- A **Big Data Analytics Platform** is an integrated solution to store, process, manage and analyze some type of Big Data.

(e.g., HDFS,
NoSQL databases)

(e.g., Hadoop,
Spark)

Organization &
governance

Analytics &
visualization

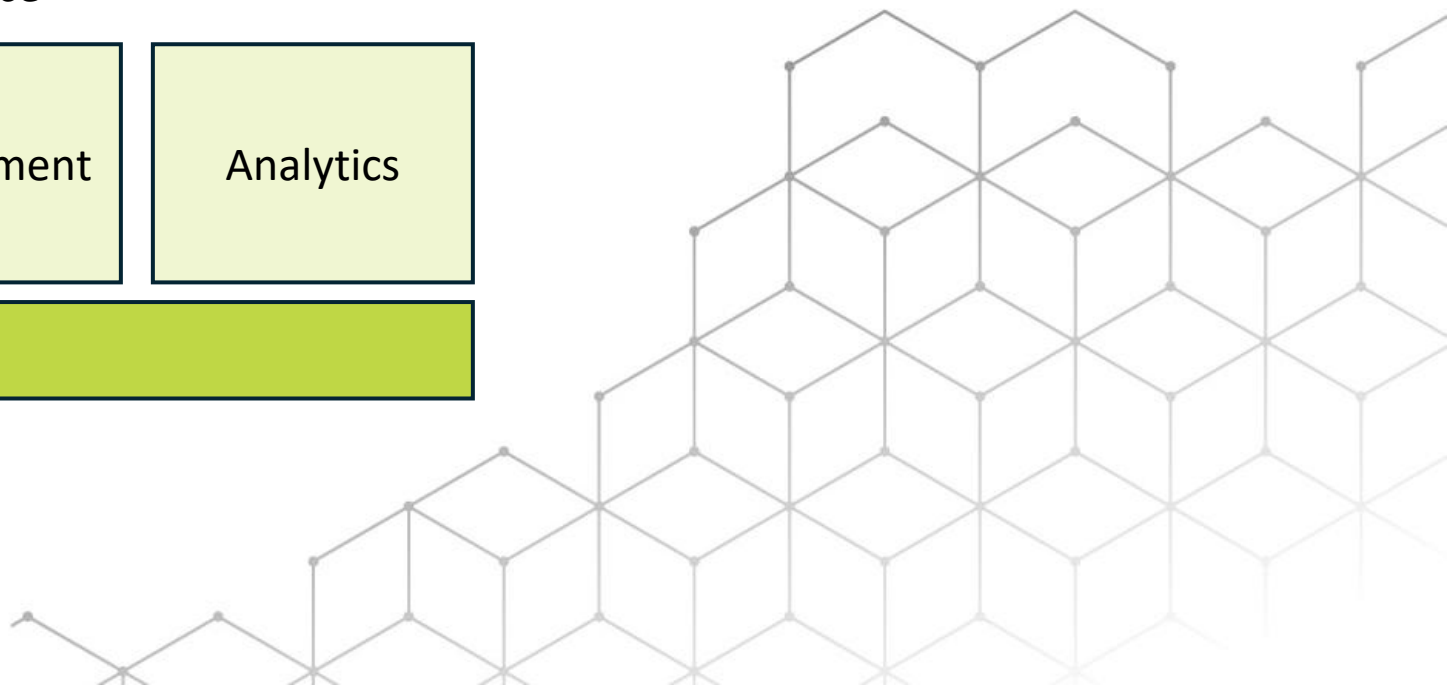
Storage

Processing

Management

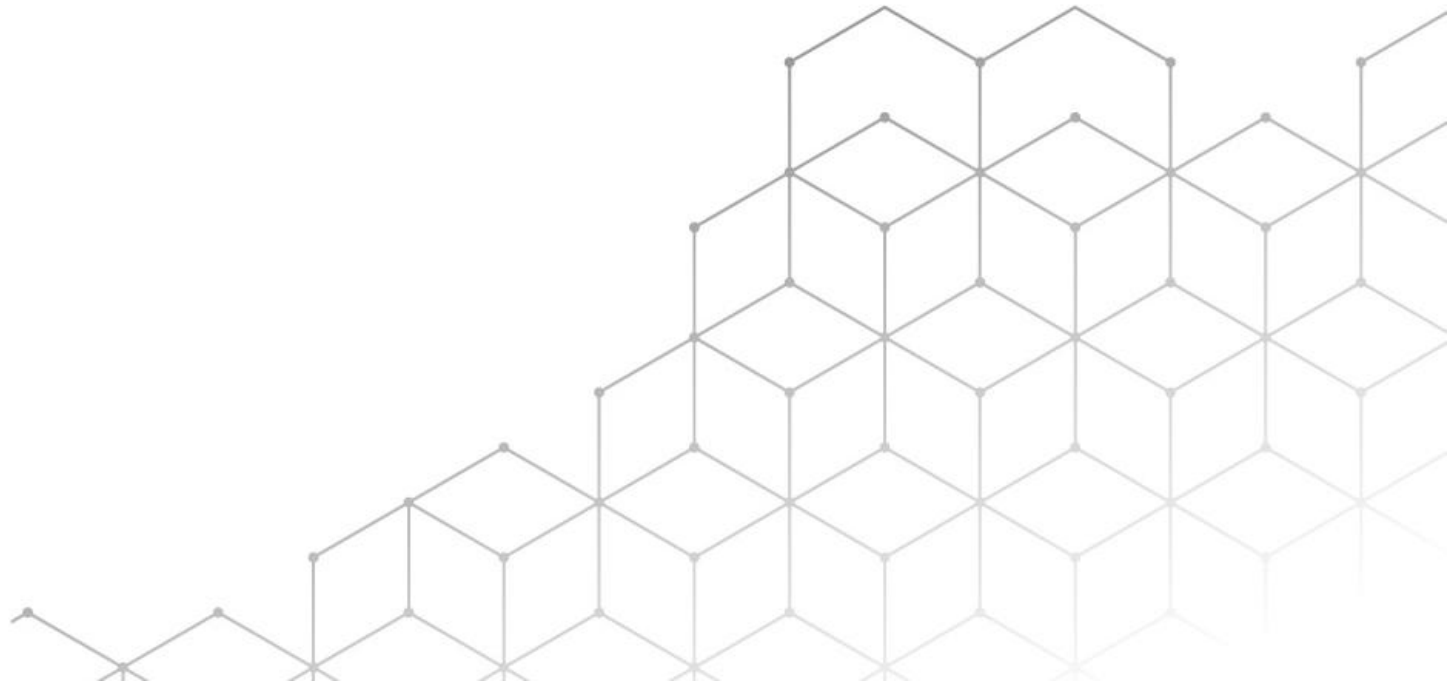
Analytics

Big Data



Learning Objectives

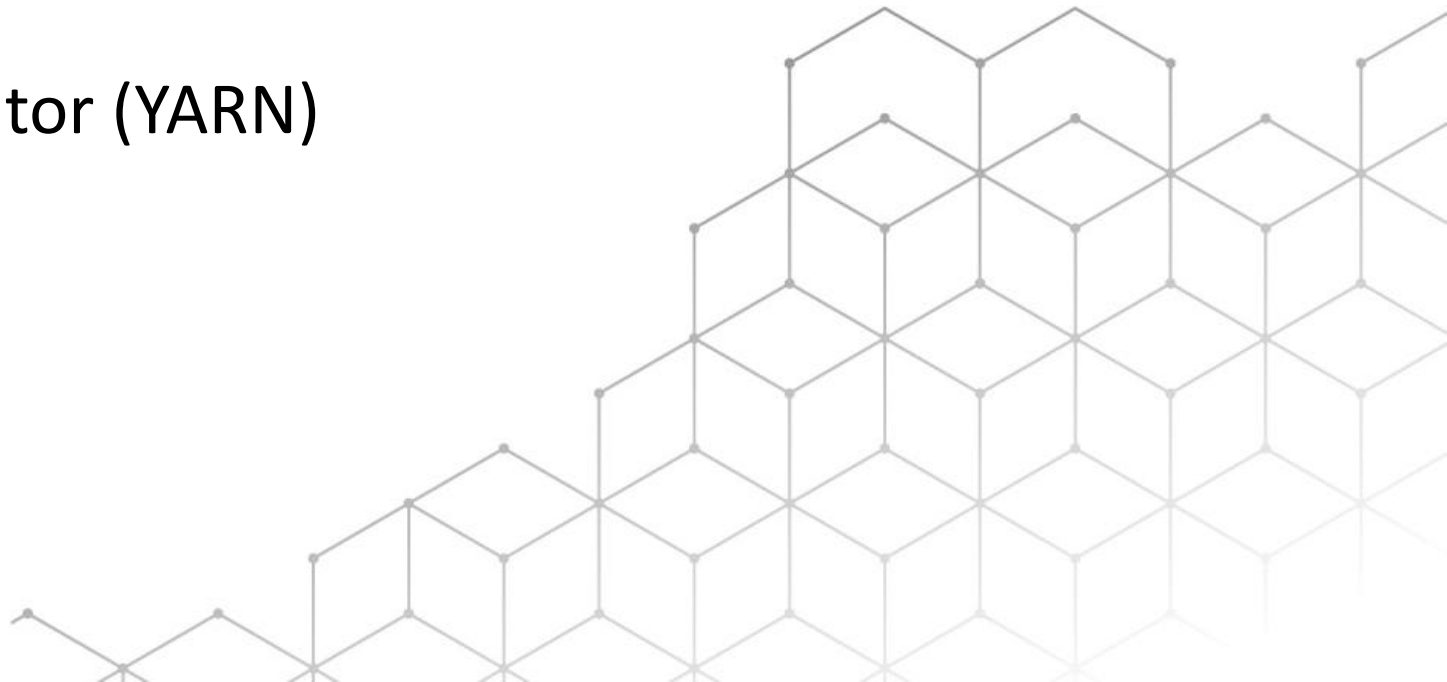
- Today we have a **busy** agenda:
 - MapReduce (Processing Framework)
 - YARN (Cluster Management)
 - Spark



Agenda for Today



- MapReduce
 - Motivation with Shakespeare Example
 - Map and Reduce
 - Mathematical Example
 - Full Example
- Yet Another Resource Navigator (YARN)
- Spark



Suppose I ask you to count how many times each word shows up in all Shakespeare books?
How would you do it?

Task



NOVA

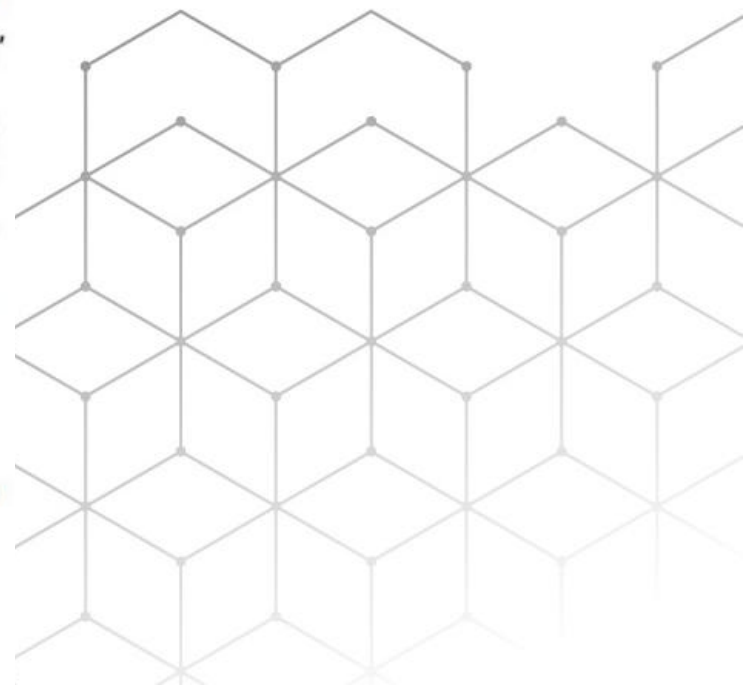
IMS

Information
Management
School

Curiosity

THINGS WE SAY TODAY, WHICH WE OWE TO SHAKESPEARE:

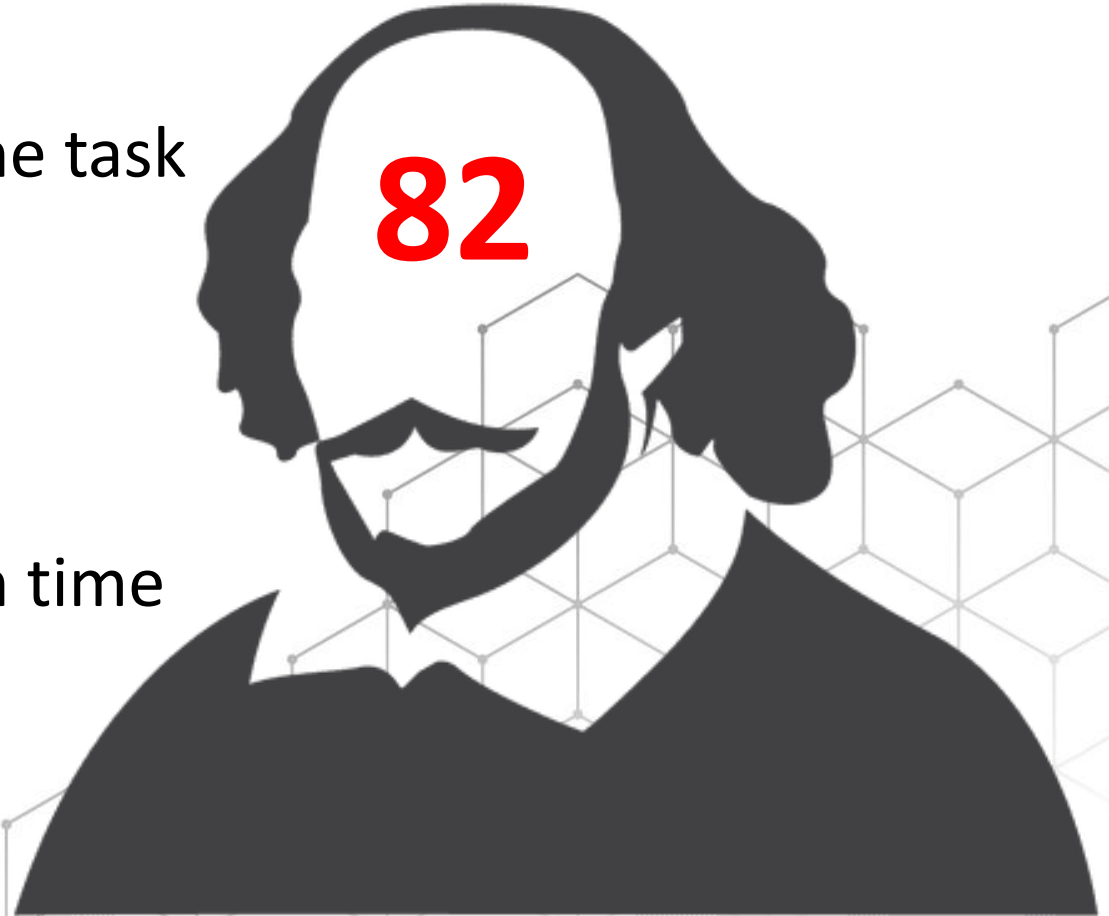
"KNOCK, KNOCK! WHO'S THERE?"
"IN A PICKLE" "SET YOUR TEETH ON EDGE" "HEART OF GOLD"
"FAINT HEARTED" "SO-SO" "GOOD RIDDANCE"
"LIE LOW" "FIGHT FIRE WITH FIRE" "BAITED BREATH" "SEND HIM PACKING"
"COME WHAT MAY"
"THE GAME IS UP"
"WEAR YOUR HEART ON YOUR SLEEVE"
"NOT SLEPT ONE WINK" "FULL CIRCLE" "OUT OF THE JAWS OF DEATH"
"WHAT'S DONE IS DONE" "NAKED TRUTH" "TOO MUCH OF A GOOD THING"
"BREAK THE ICE"
"LAUGHING STOCK" "BREATHED HIS LAST" "WILD GOOSE CHASE"
"HEART OF HEARTS" "VANISH INTO THIN AIR"
"SEEN BETTER DAYS" "MAKES YOUR HAIR STAND ON END"
"DEAD AS A DOORNAIL" "FOR GOODNESS' SAKE" "LOVE IS BLIND"
"GREEN EYED MONSTER" "FAIR / FOUL PLAY / PLAY" "OFF WITH HIS HEAD"
"THE WORLD IS MY OYSTER" "BRAVE NEW WORLD"
"BE ALL / END ALL" "A SORRY SIGHT"



Embarrassingly Simple...

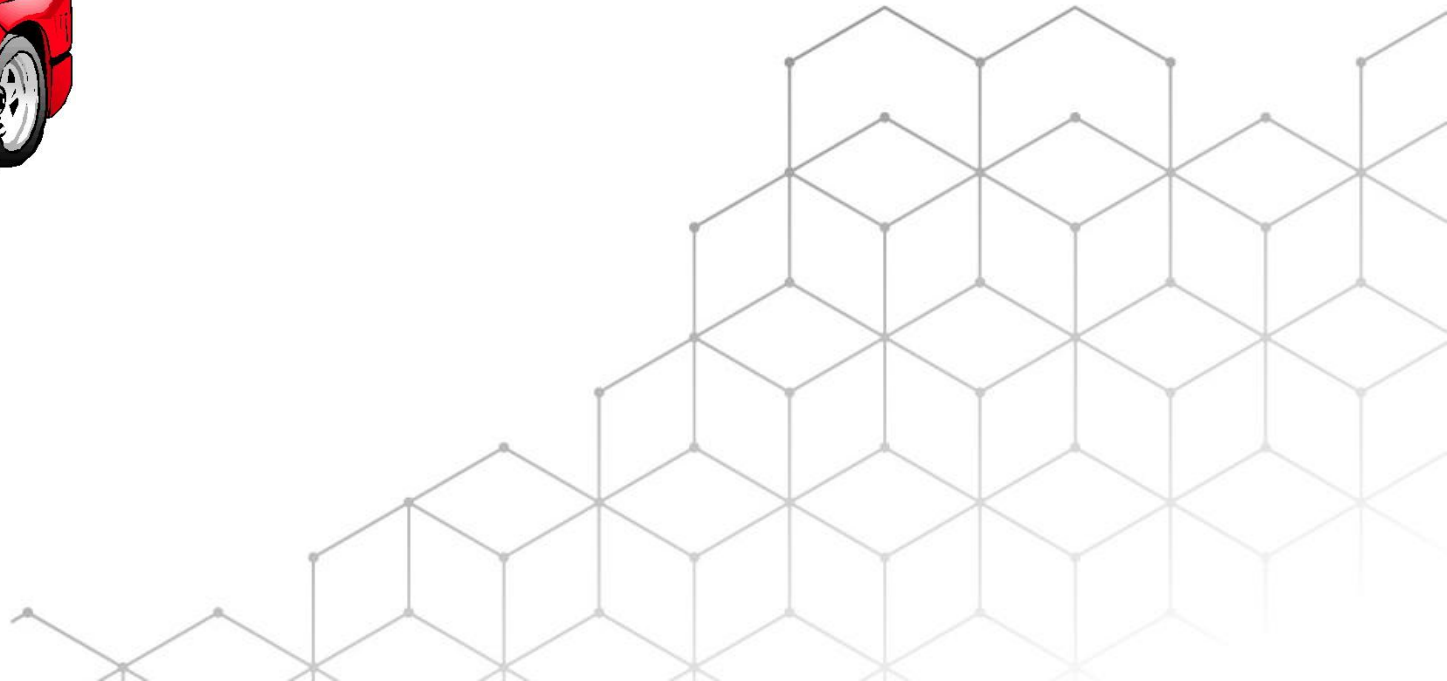
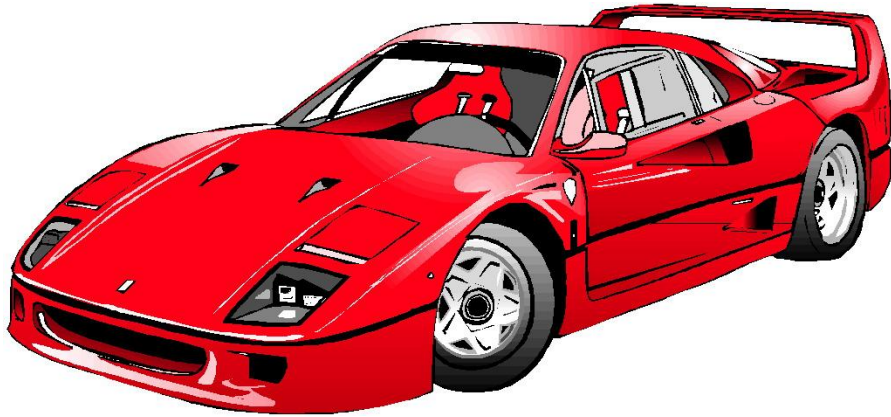
- since Shakespeare wrote 118 406* lines of text
- it would take you **118 406** minutes
- or 1973.4 hours (~ **82** days) to finish the task

that is, if excel doesn't crash in the mean time



Speed

- Can we generalize a framework to speed up tasks similar to this one?



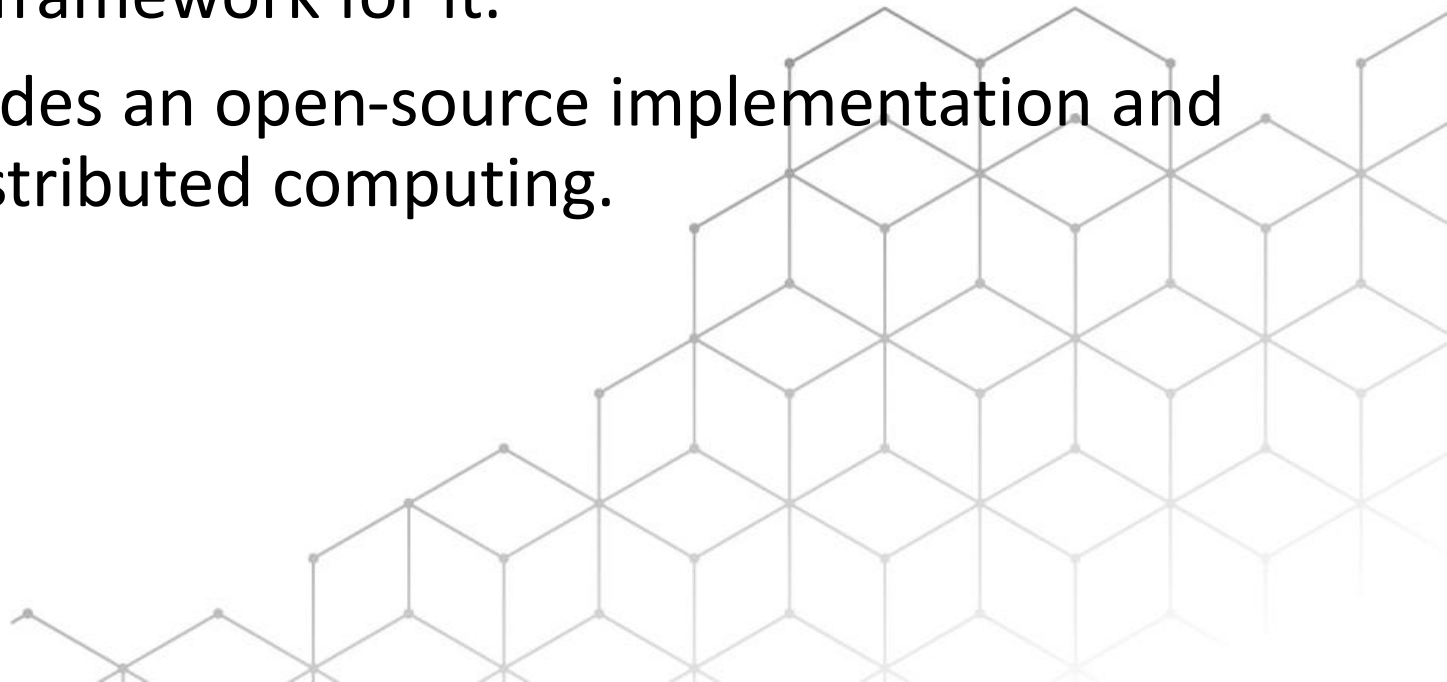
Going Back to the Example

- Like in our previous example, if you have N pages (N very large)
- On a single machine, this process takes $O(N)$ time
- However, this is (almost) embarrassingly parallel
 - Whether any word appears in a document is independent of other documents
 - We should process documents independently and combine results.



Solution

- We could have multiple computers write to a shared database
- Can we lower the time to $O(N/M)$ with M machines?
- How to distribute work (and data) and collect results?
- Map and Reduce provides a framework for it.
- As we will see, Hadoop provides an open-source implementation and infrastructure support for distributed computing.



Map and Reduce

- Map and Reduce are common second-order functions in functional programming
- **Second-order functions** are functions that **take other functions as arguments** or return functions as results.

map: function, list $\rightarrow$ list

$\text{map}(\text{function } f, [x_1, x_2, \dots, x_n]) \rightarrow [f(x_1), f(x_2), \dots, f(x_n)]$

reduce: function, list $\rightarrow$ item

$\text{reduce}(\text{function } g, [x_1, x_2, \dots, x_n]) \rightarrow g(x_1, \text{reduce}(g, [x_2, \dots, x_n]))$



Example of Map Reduce

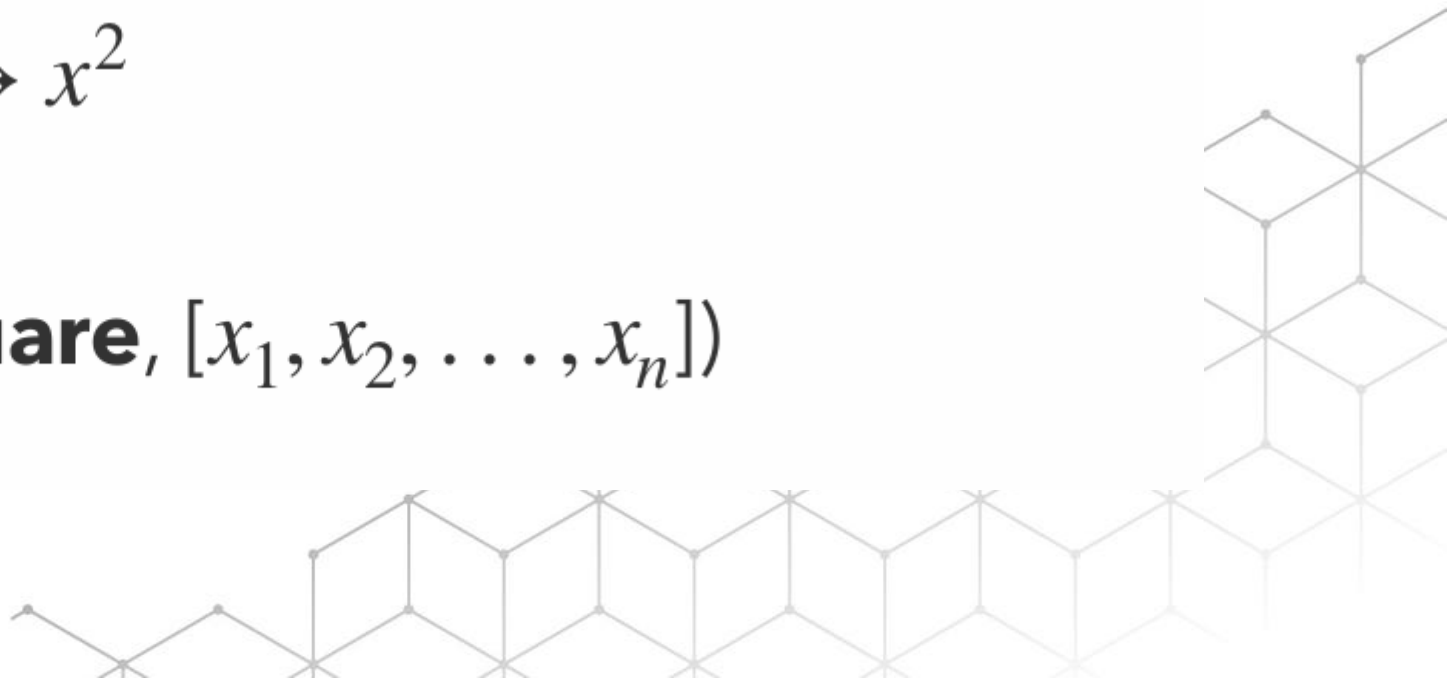
Define functions sum and square

$$\mathbf{sum}: x, y \rightarrow x + y$$

$$\mathbf{sum}: [] \rightarrow 0$$

$$\mathbf{square}: x \rightarrow x^2$$

$$\mathbf{reduce}(\mathbf{sum}, \mathbf{map}(\mathbf{square}, [x_1, x_2, \dots, x_n]))$$



Example Solved

`reduce(sum, map(square, [1,2,3,4,5]))`

→ `reduce(sum, [square(1), square(2), square(3), square(4), square(5)])`

→ `reduce(sum, [1,4,9,16,25])`

→ `(1+(4+(9+(16+(25))))))`

→ 55



load reduce function →

```
from functools import reduce
import random
```

create toy list →

```
numbers = [random.random() for i in range(100)]
```

mapper →

```
def square(x):
    return x*x
```

reducer →

```
def add(x,y):
    return x+y
```

check list →

```
print(numbers[1:5])
```

```
[0.6502014454185554, 0.38855263482835867, 0.956984892631307, 0.8308572352347688]
```

execute map stage →

```
mapstage = map(square,numbers)
```

```
print(list(mapstage)[1:5])
```

```
[0.4227619196243787, 0.15097315003205983, 0.9158200847245542, 0.6903237453419641]
```

execute reduce stage →

```
reducestage = reduce(add,mapstage)
```

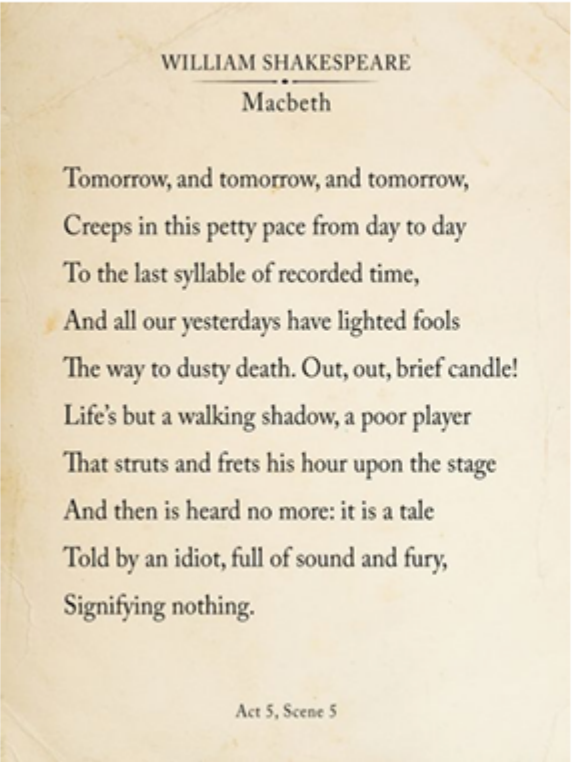
```
print(reducestage)
```

```
35.85521892654224
```



Shakespeare Example: The Setup

Input Data



MAP

REDUCE

Given a file text, we want to write two functions: a Map and a Reduce, that allow us to count the number of times each word shows up in that text.

Result

	A	B
1	Count of Word	
2	Word	Total
3	dup	11
4	dos	9
5	reject	8
6	da	8
7	same	7
8	multiple	6
9	day	6
10	visits	6
11	claim	4
12	req	3
13	denied	2
14	call	2
15	sandi	2
16	called	2
17	pt	2
18	tt	2
19	rekey	2
20	acct	2

Map Function

"ORDINARIAMENTE todos os ministros são inteligentes, escrevem bem, discursam com cortesia e pura dicção, vão a jausiosas inaugurações e sao excelentes convivas. Porém, são nulos a resolver crises. Nao tem a austeridade, nem a concepção, nem o instinto politico, nem a experiência que faz o ESTADISTA. E assim que há muito tempo em Portugal são regidos os destinos políticos. Política de acaso, política de compadrio, política de expediente. País governado ao acaso, governado por vaidades e por interesses, por especulação e corrupção, por privilégio e influência de camarilha, será possível conservar a sua independência?"

(Eça de Queiroz, 1867 in "O distrito de Évora")

Map Function takes as input each line of text:

- parse text (lower case, remove punctuation, etc ..)
- break it into a sequence of words
- returns a sequence of (word, token)

Mapper Output

(ordinariamente,1)
(todos,1)
(os,1)
(ministros,1)
(sao,1)
(inteligentes,1)

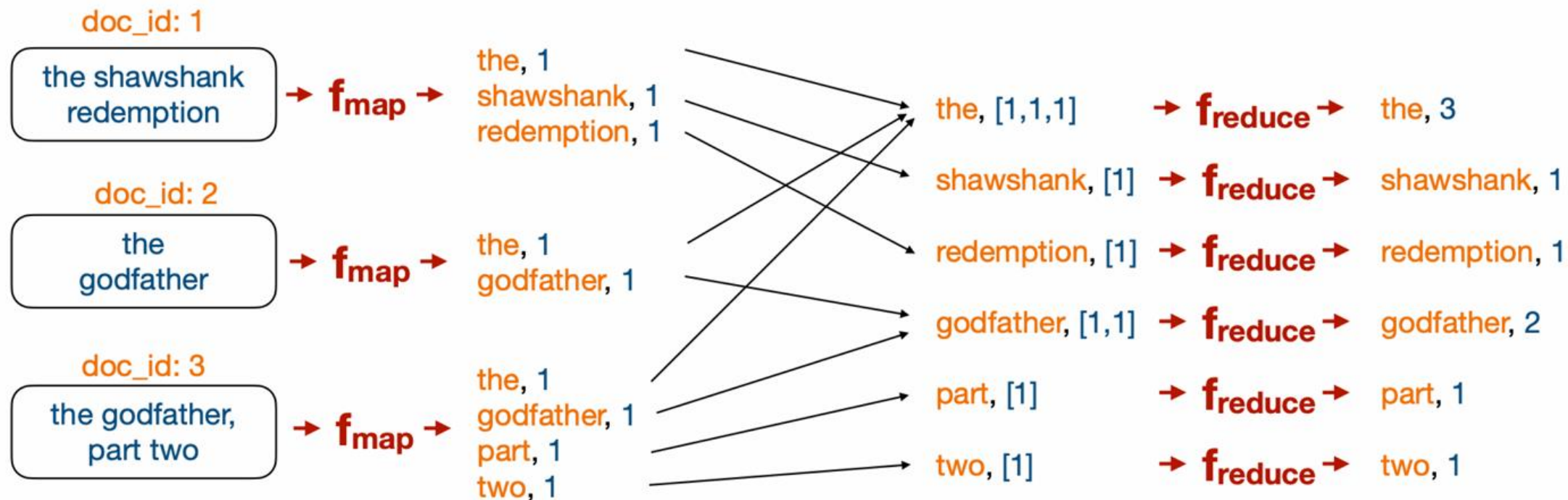
(escrevem,1)
(bem,1)
(discursam,1)
(com,1)
(cortesia,1)
(e,1)
(pura,1)
(diccao,1)
(vao,1)
(a,1)

(faustosas,1)
(inaugurações,1)
(e,1)
(sao,1)
(excelentes,1)
(convivas,1)
(porem,1)
(sao,1)

...



Full Example



Map-Phase

```
fmap (doc_id, doc_text):
    for word in doc_text:
        emit word, 1
```

Shuffle-Phase

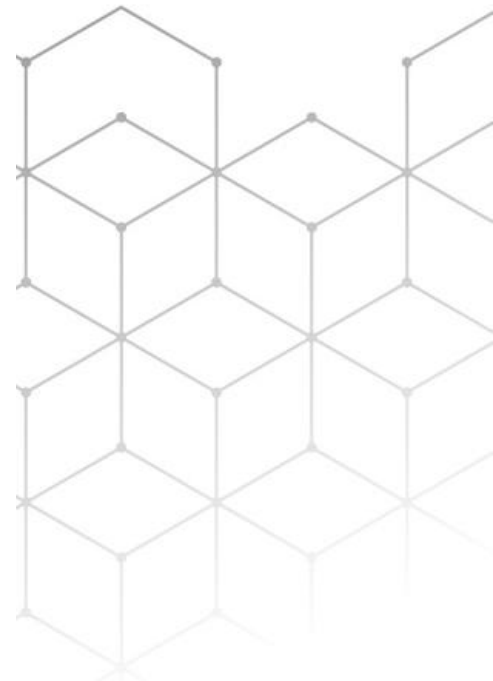
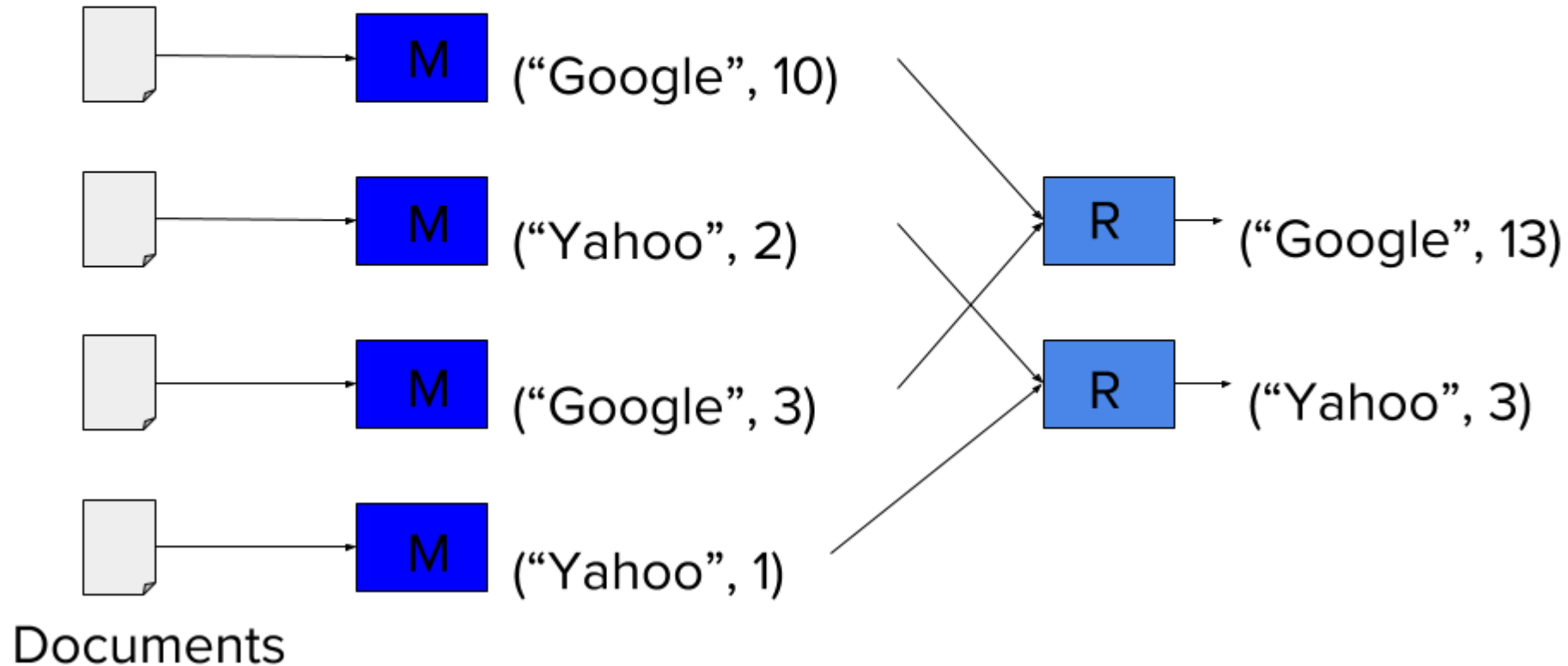
Reduce-Phase

```
freduce (word, counts):
    total = 0
    for count in counts:
        total += count
    emit word, total
```

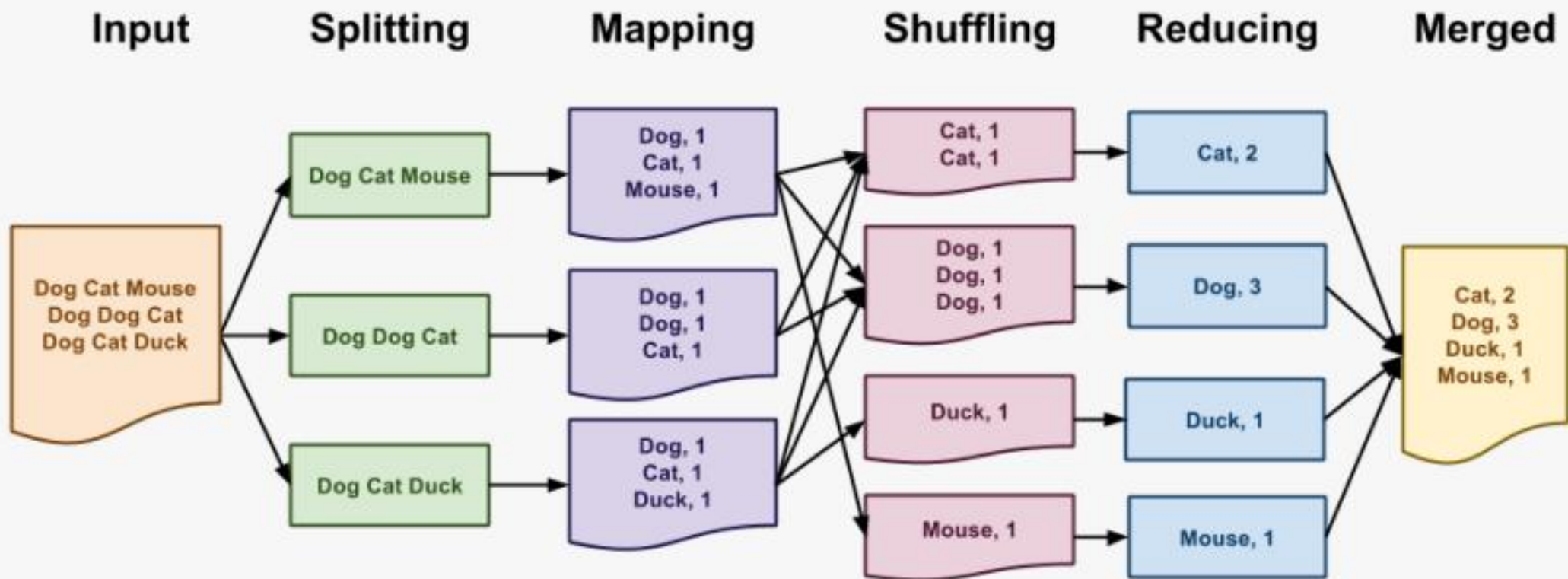
Simpler Example

Map()

Reduce()



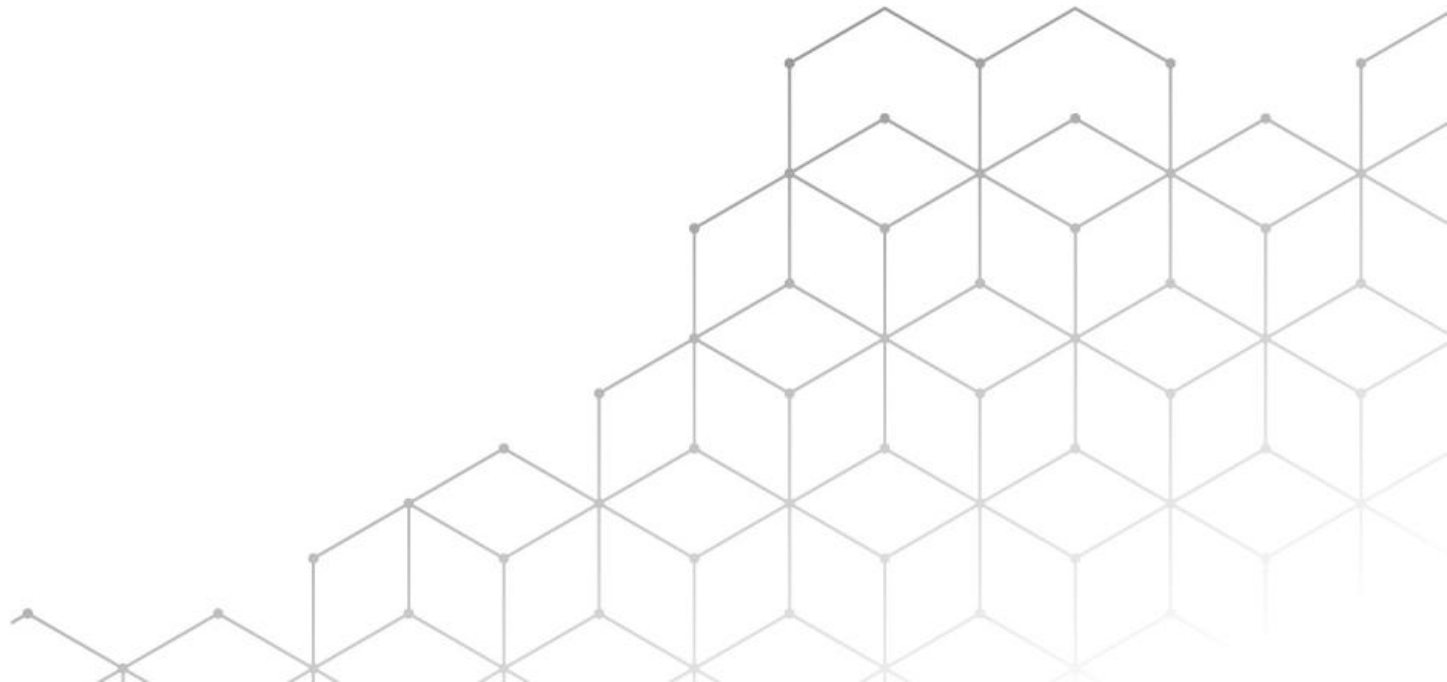
Another Example



Agenda for Today

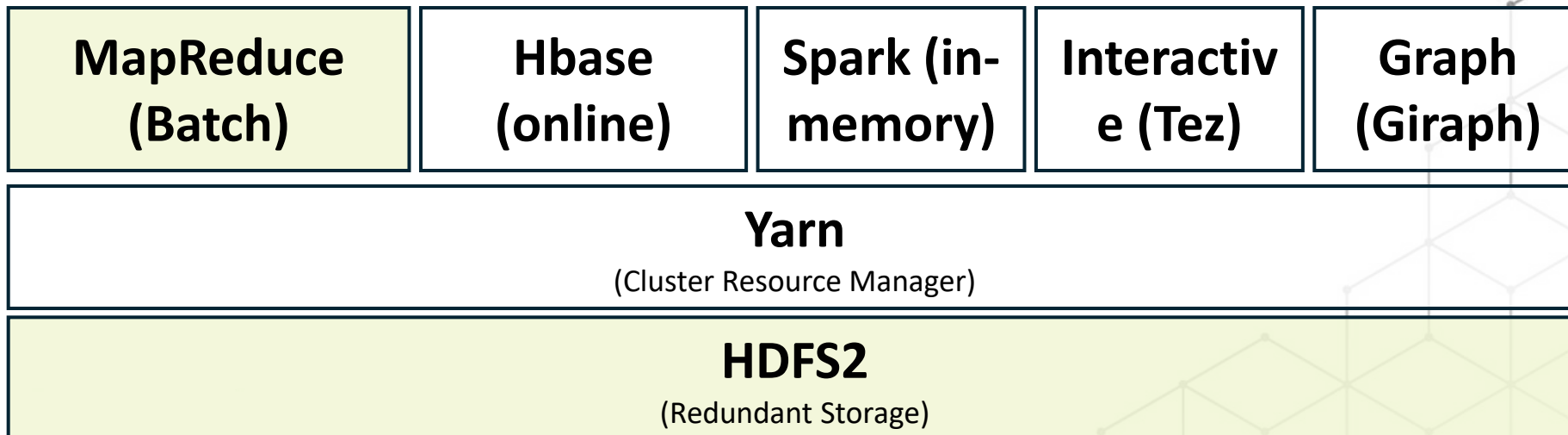


- MapReduce
- Yet Another Resource Navigator (YARN)
 - Yet Another Resource Navigator
 - MapReduce Execution
 - MapReduce vs YARN
 - Yarn Execution
 - Example
- Spark

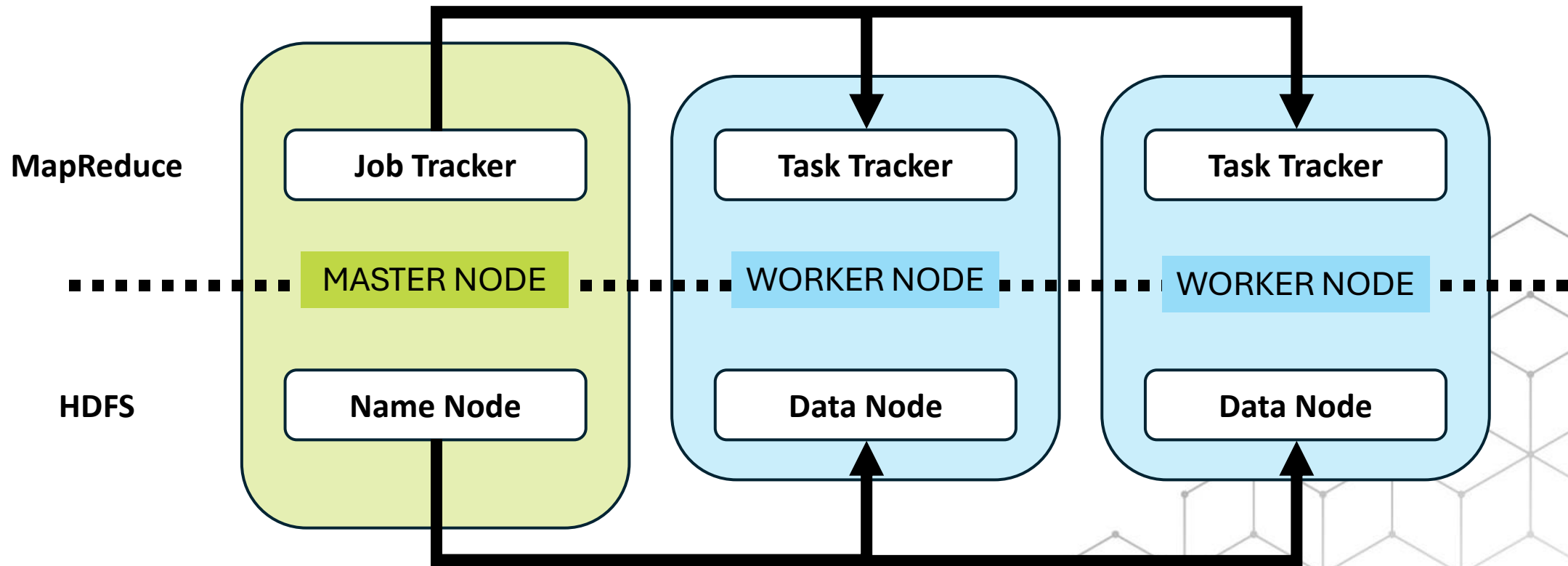


Yet Another Resource Navigator (YARN)

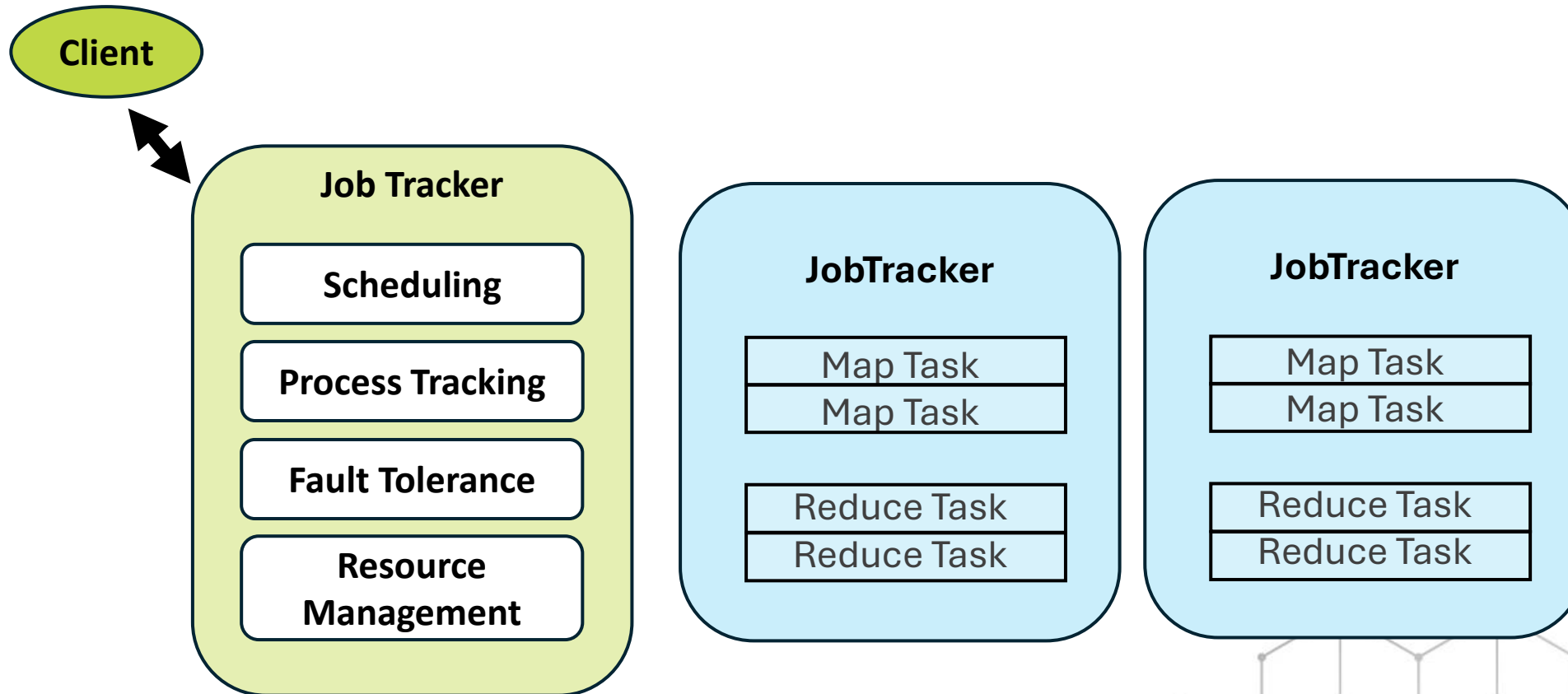
- Component to enable other processing frameworks to run on Hadoop.
- Hadoop went from a batch processing engine into a platform for many different modes of data processing.



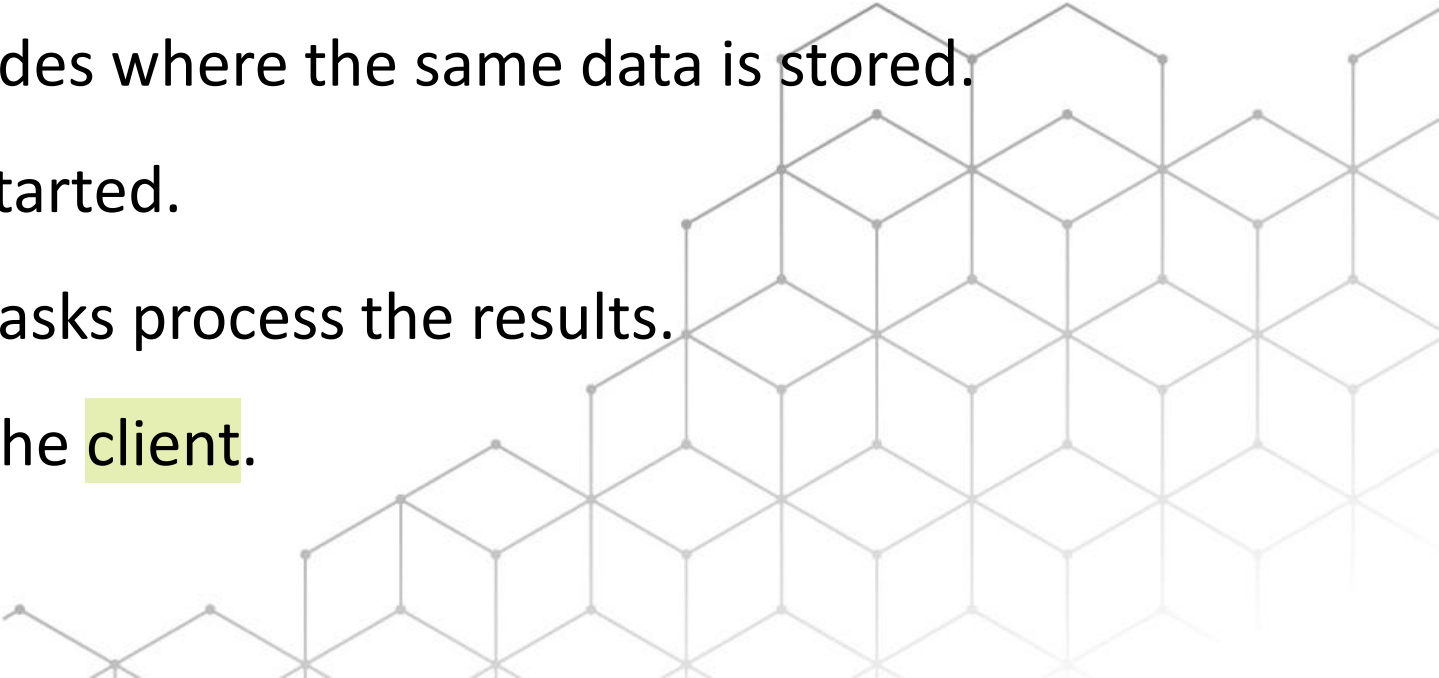
MapReduce Execution (1)



MapReduce Execution (2)



MapReduce Execution Description

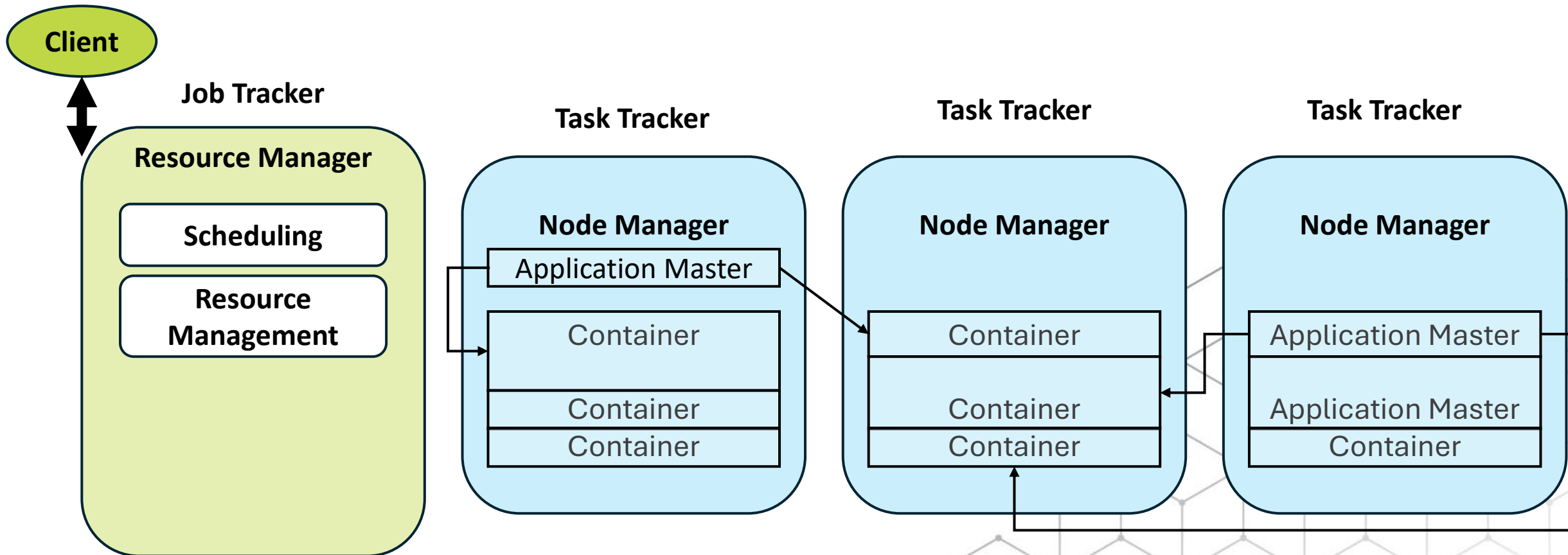
- 1) The **client** makes an **application** request to **JobTracker**.
 - 2) The **JobTracker** determines processing resources for the application.
 - 3) The **JobTracker** queues all the **map** tasks and **reduce** tasks for execution.
 - 4) Map tasks are assigned to nodes where the same data is stored.
 - 5) If node failure, the task is restarted.
 - 6) After the **map** tasks, **reduce** tasks process the results.
 - 7) The result set is returned to the **client**.
- 

MapReduce vs YARN

- MapReduce is a successful batch-oriented programming model.
- Demand for additional processing modes:
 - Graph Analysis
 - Stream data processing
 - Message passing
- YARN helped move beyond these limitations



Yarn Execution (1)

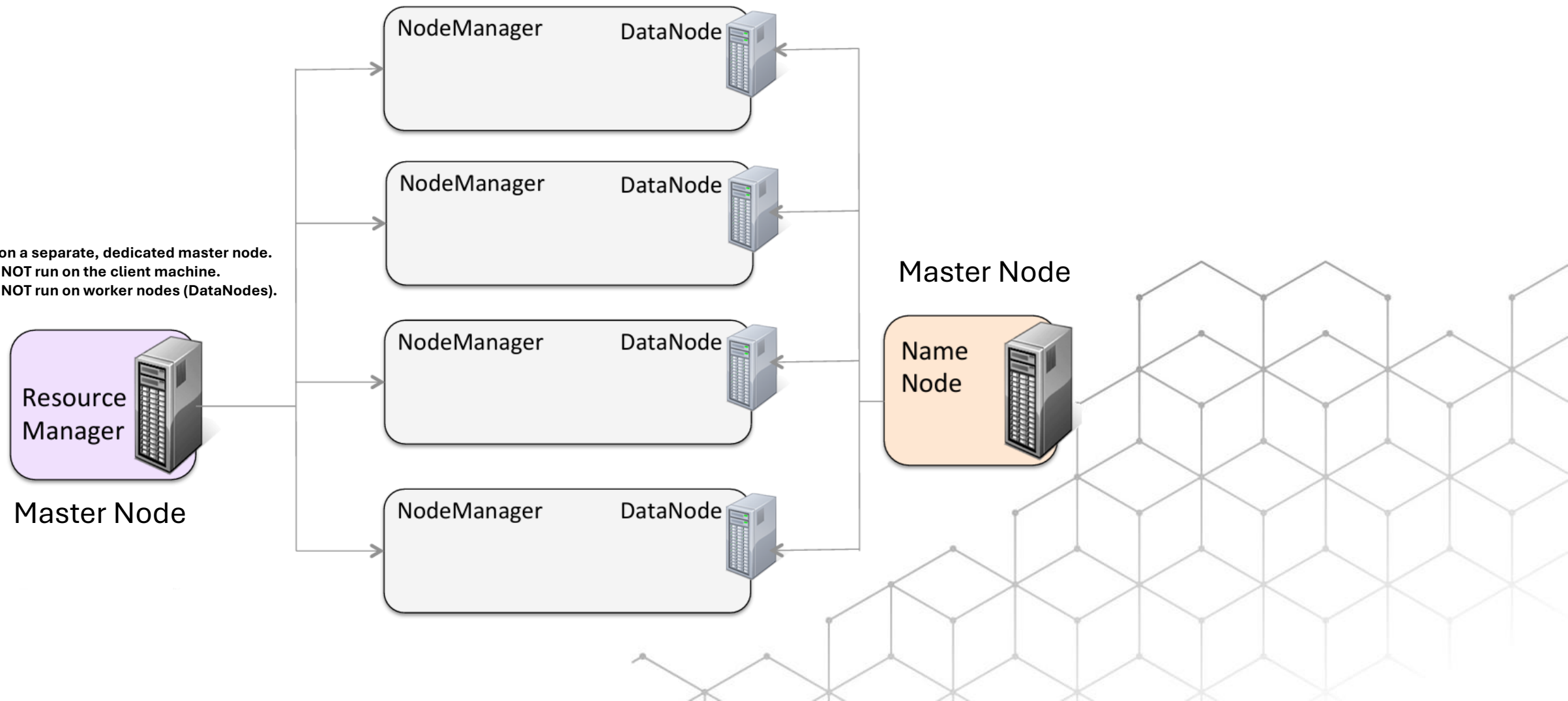


Yearn Execution Description

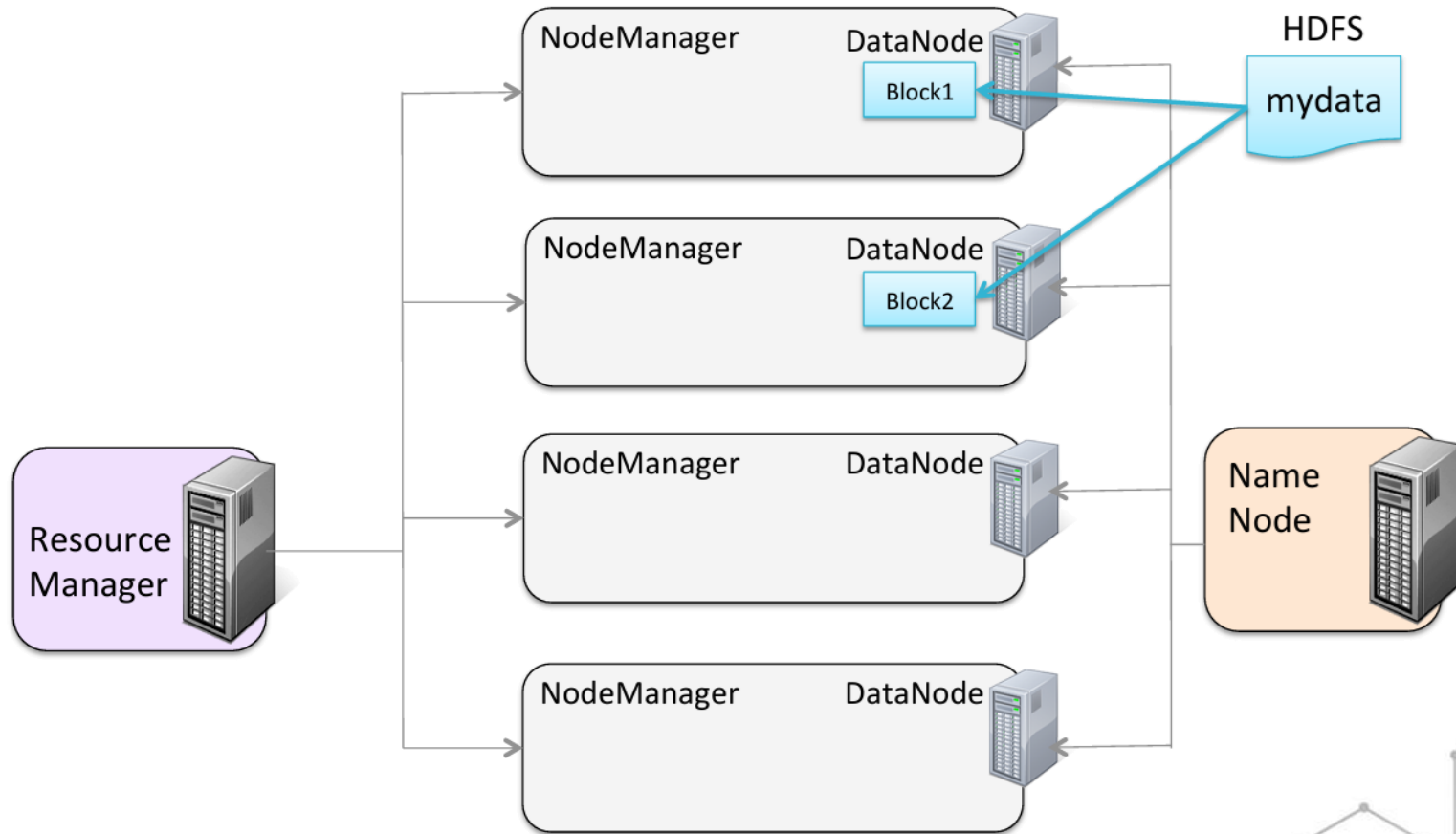
- 1) Client submits an application to Resource Manager.
- 2) Resource Manager asks a Node Manager to create an Application Master instance and start up.
- 3) The Application Master requests the resources from the Resource Manager
- 4) The Resource Manager accepts the request and queue up.
- 5) As the requested resources become available on the slave nodes, the Resource Manager grants the Application Master leases for containers on specific slave nodes.

Yarn in Detail

- ✓ It runs on a separate, dedicated master node.
- ✓ It does NOT run on the client machine.
- ✓ It does NOT run on worker nodes (DataNodes).

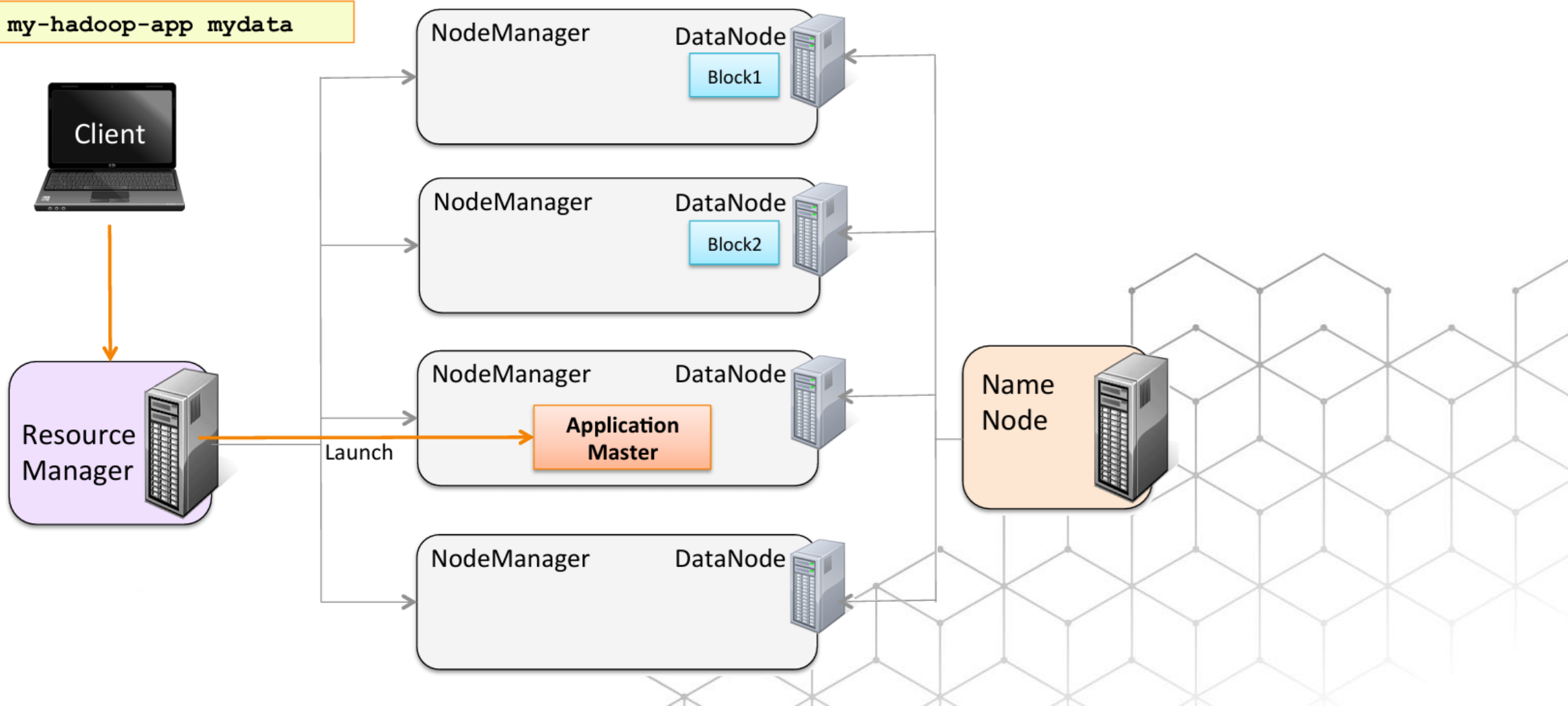


Example (1)



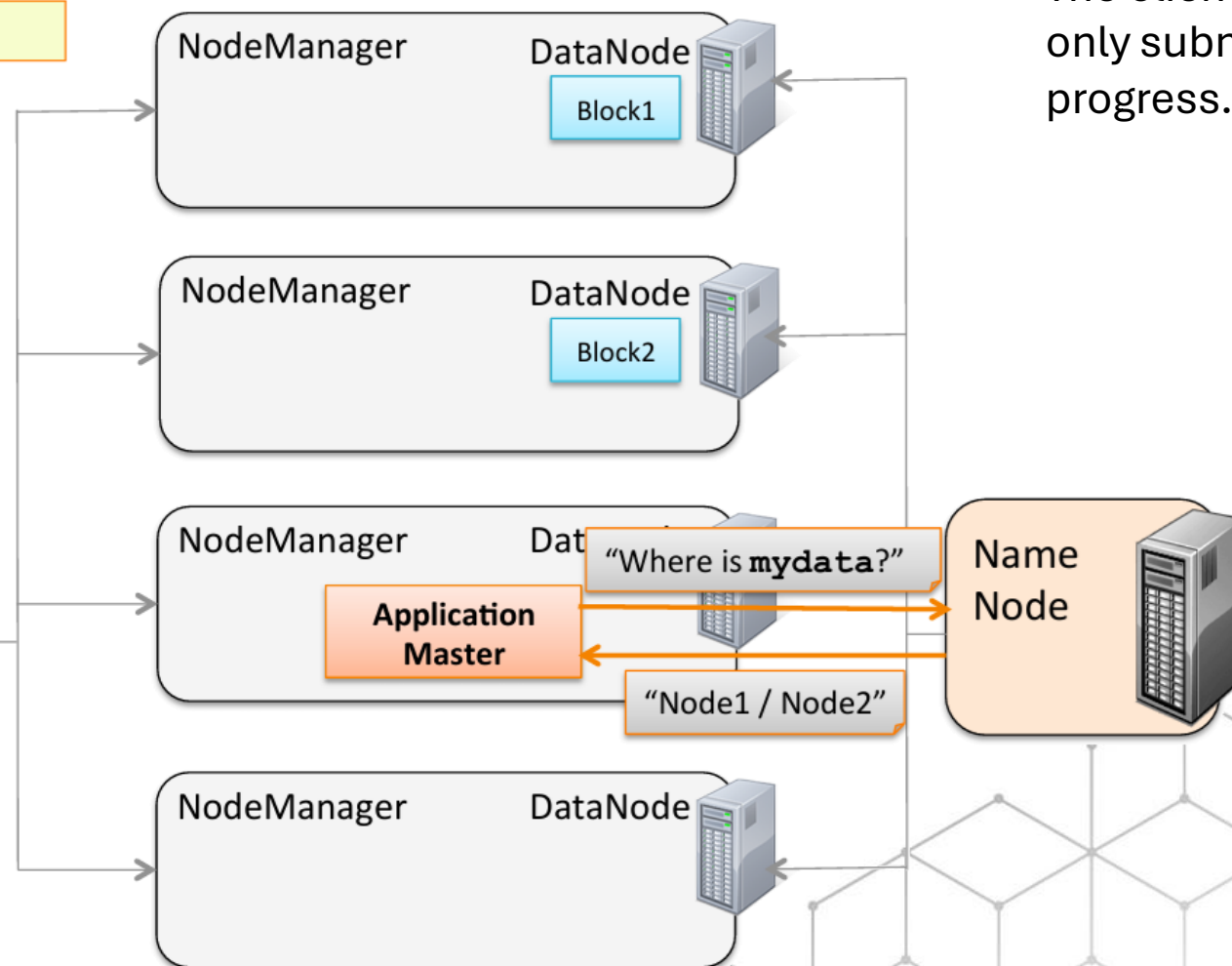
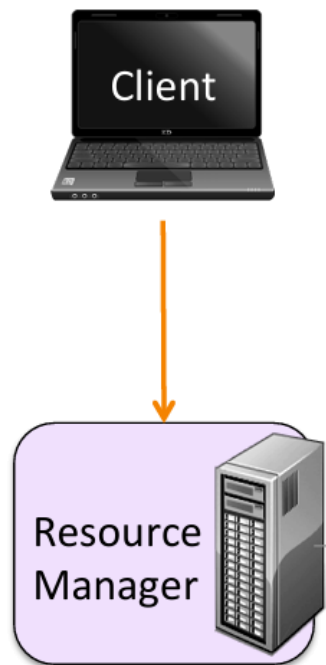
Example (2)

```
$ my-hadoop-app mydata
```



Example (3)

```
$ my-hadoop-app mydata
```

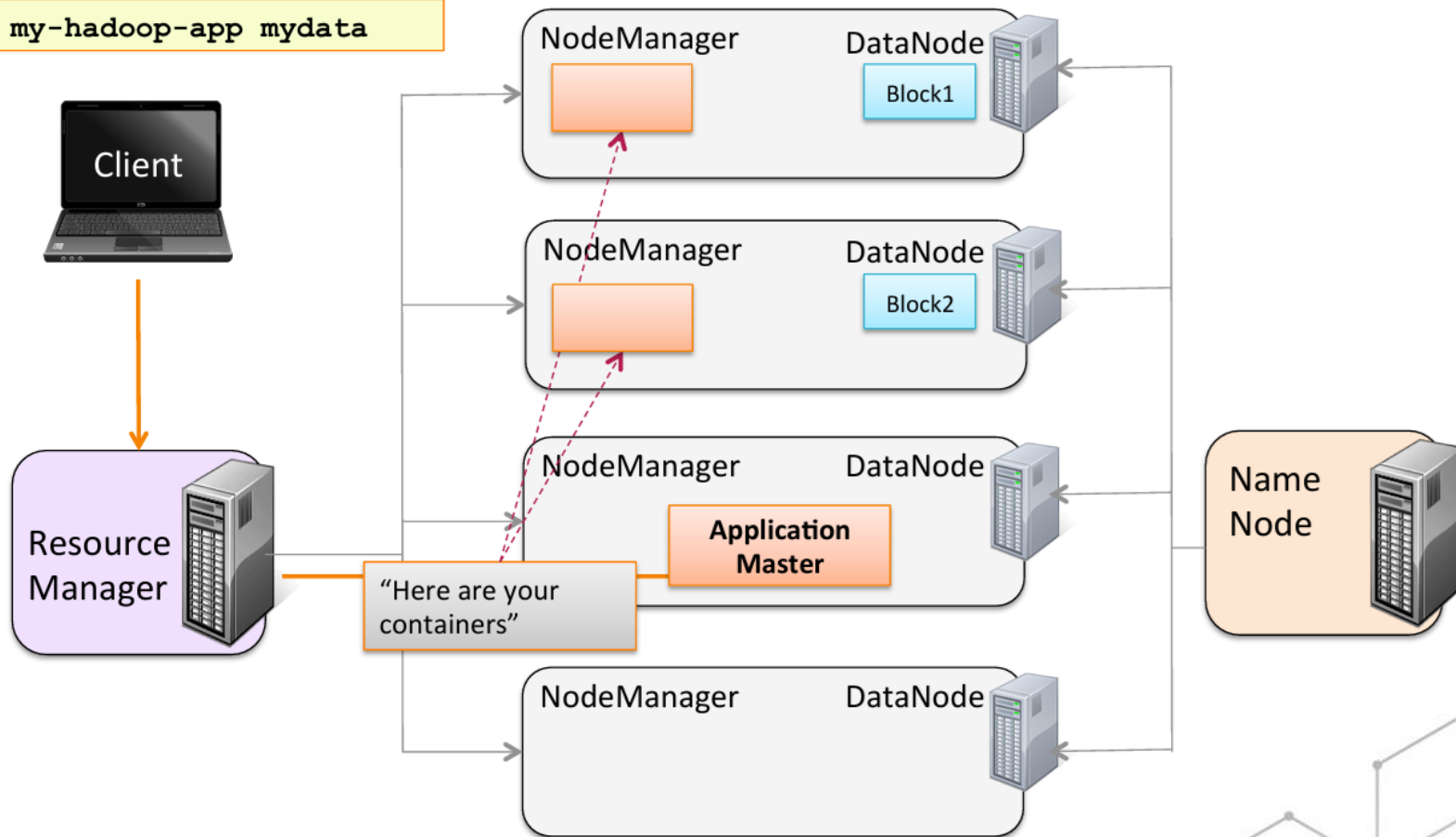


The client machine (user's machine) submits jobs to the ResourceManager.

The client does not execute jobs—only submits them and monitors progress.

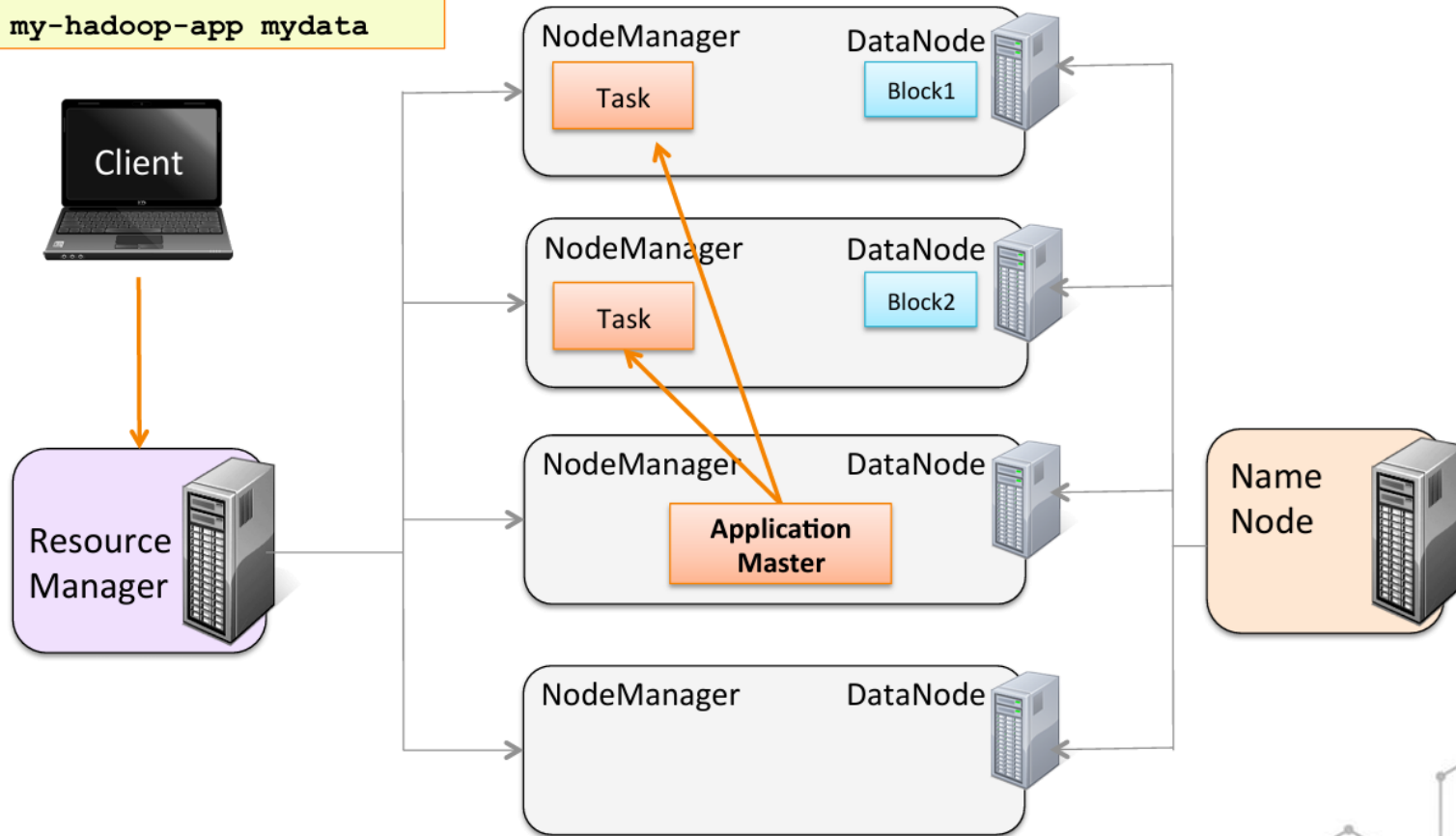
Example (4)

```
$ my-hadoop-app mydata
```



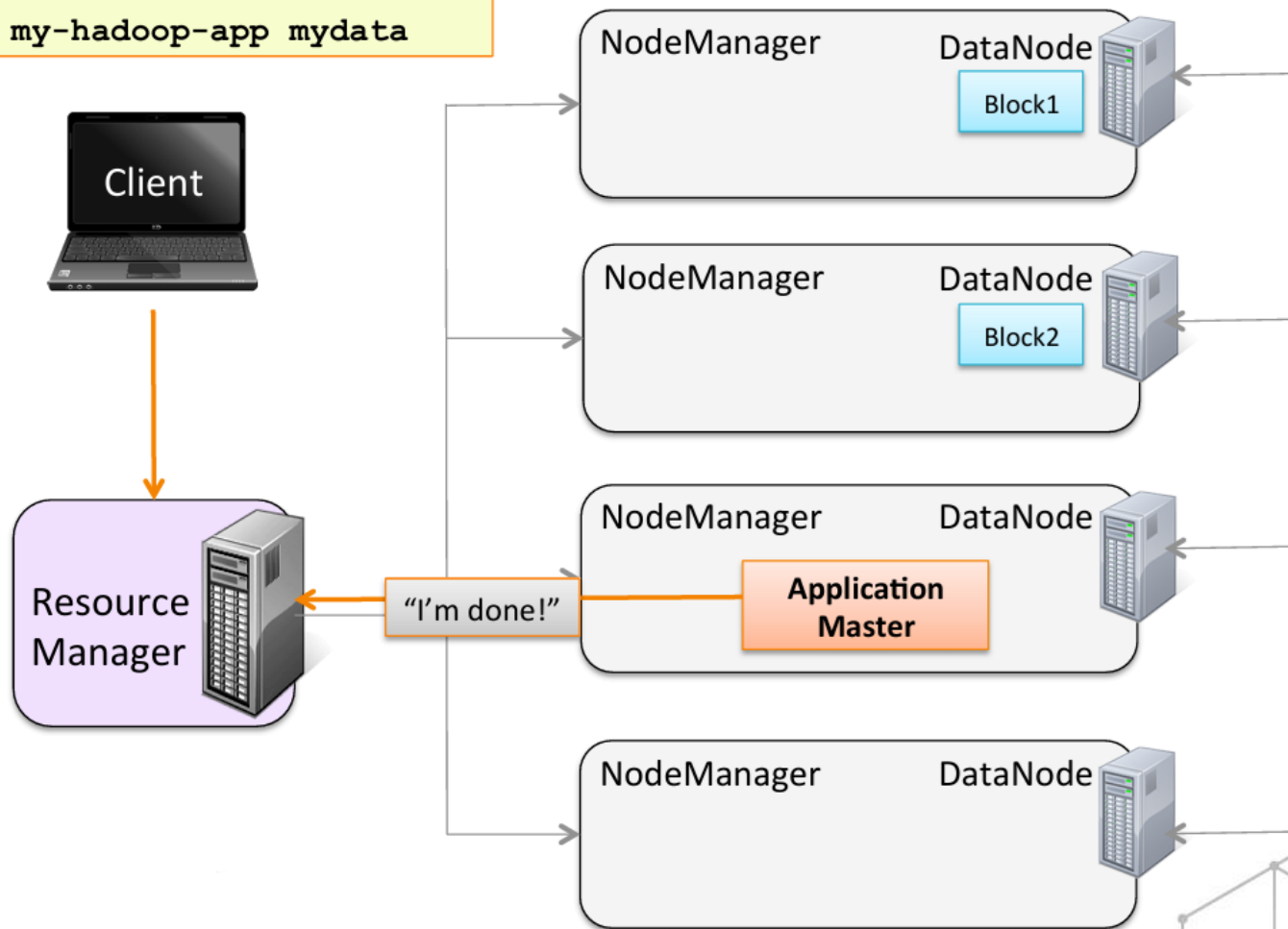
Example (5)

```
$ my-hadoop-app mydata
```



Example (6)

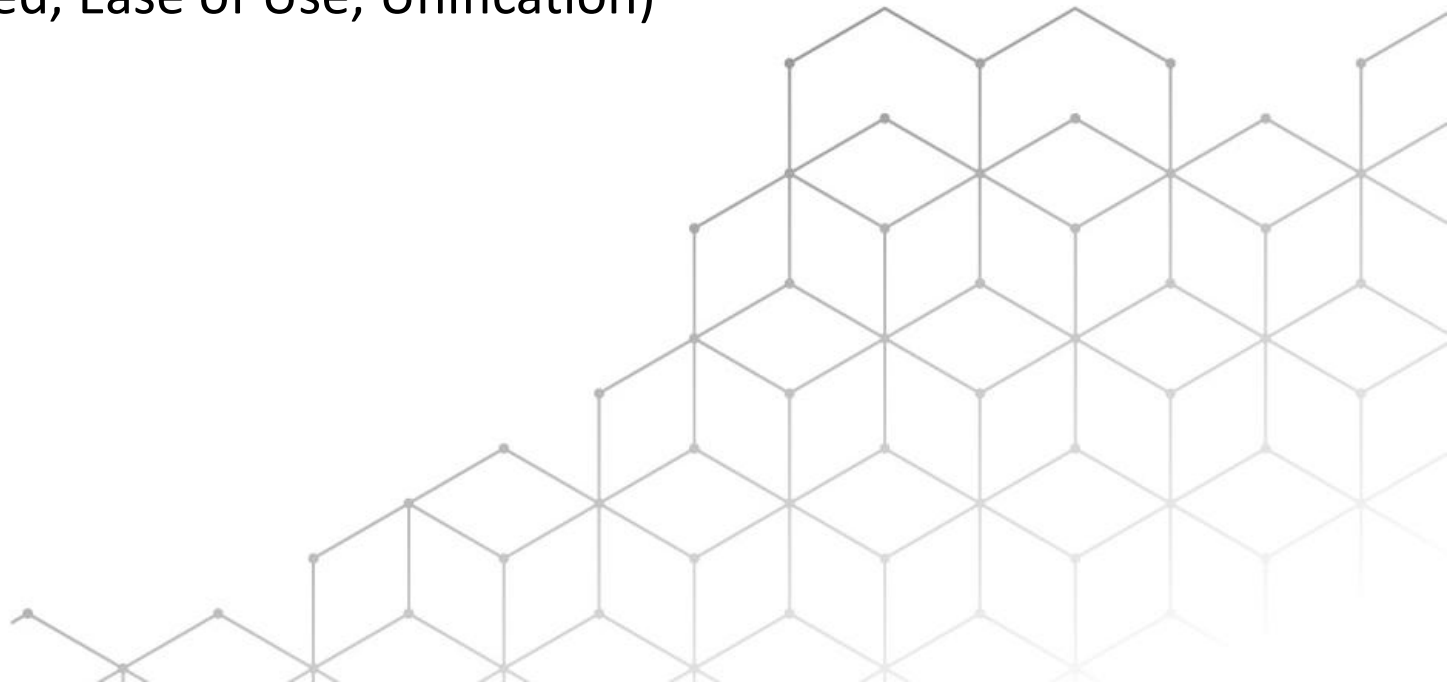
```
$ my-hadoop-app mydata
```



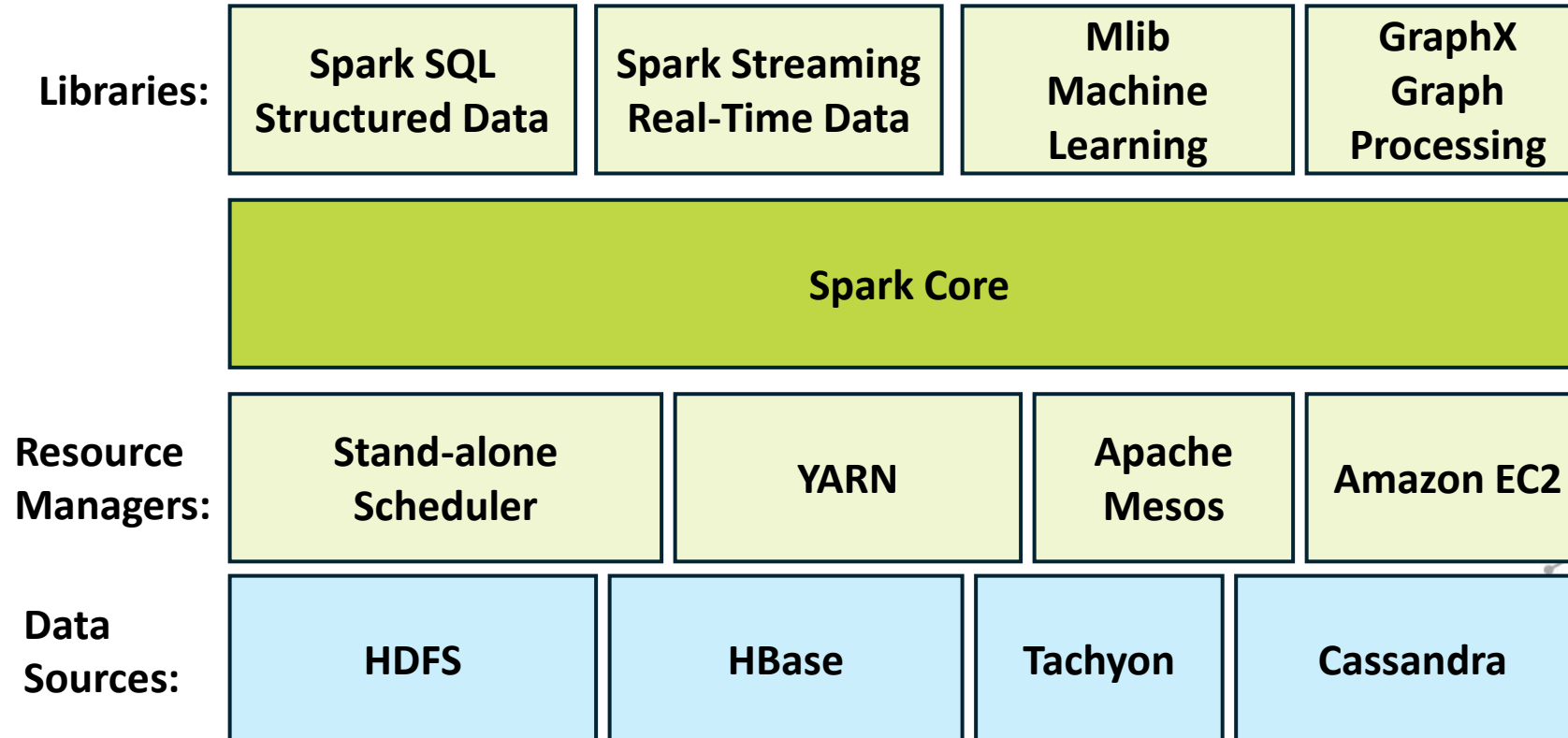
Agenda for Today



- MapReduce
- Yet Another Resource Navigator (YARN)
- Spark
 - Architecture of Spark
 - Properties of Apache Spark (Speed, Ease of Use, Unification)
 - Use Cases
 - Advantages of Apache Spark
 - Spark Driver Program
 - Execution Plan
 - Core Spark Concepts
 - Workflow of Spark Application
 - Disadvantages of Spark

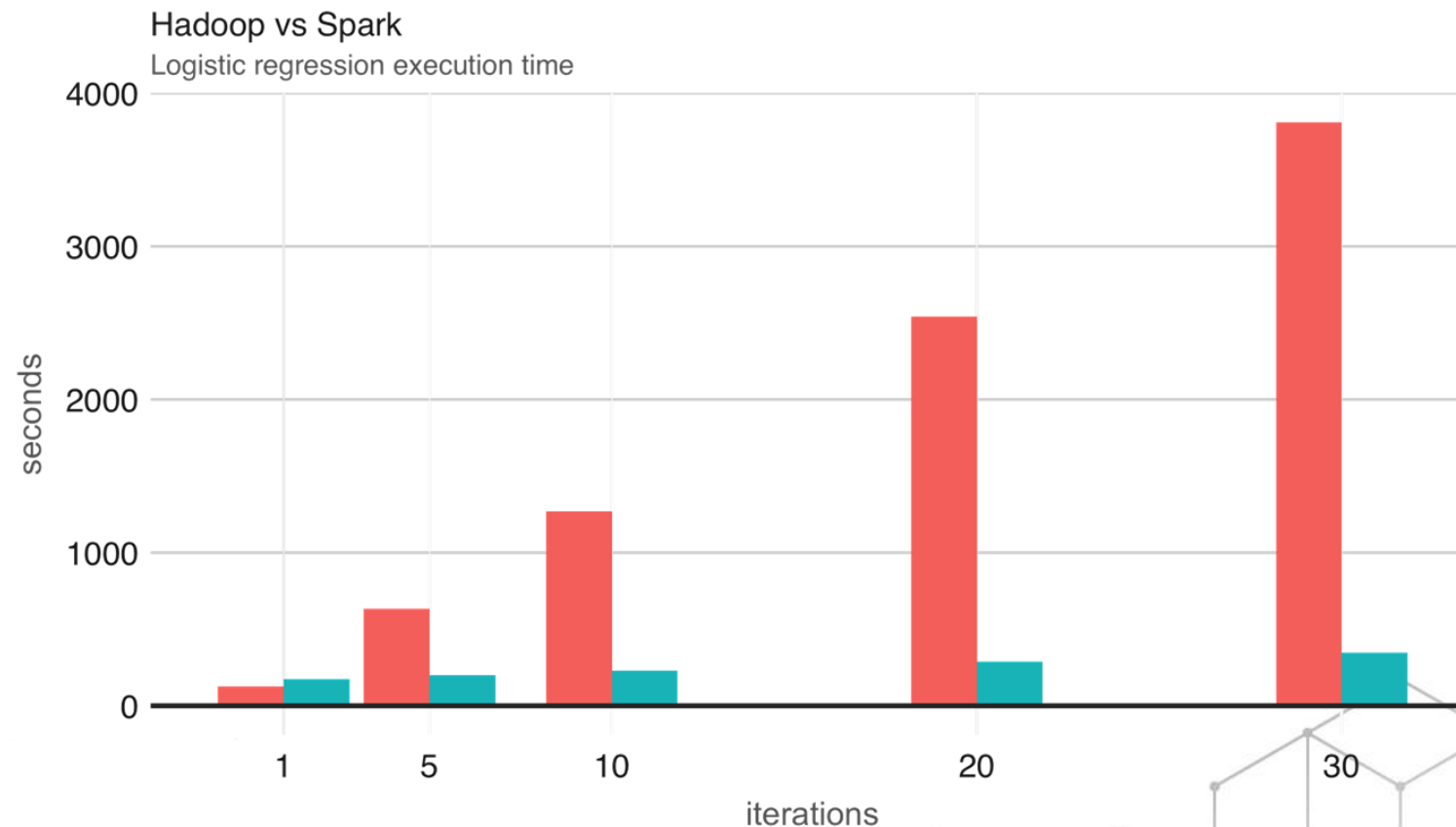


Architecture of Spark



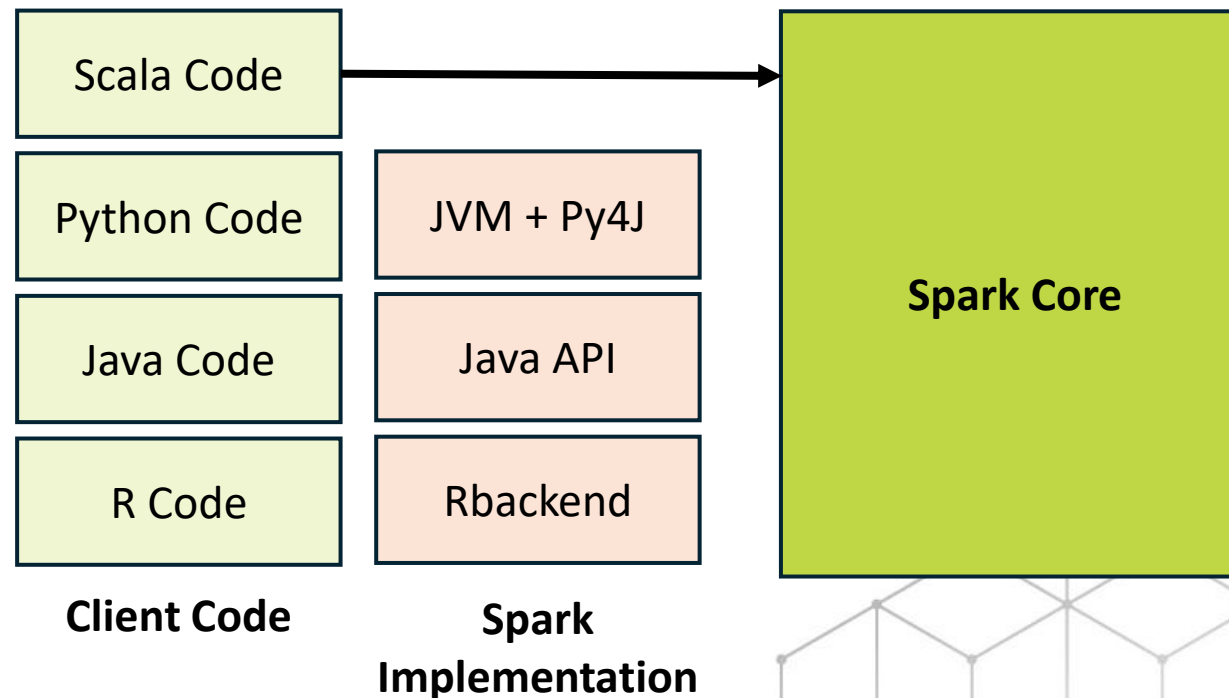
Speed of Apache Spark

- Speed: Processes data **much faster** than traditional MapReduce.



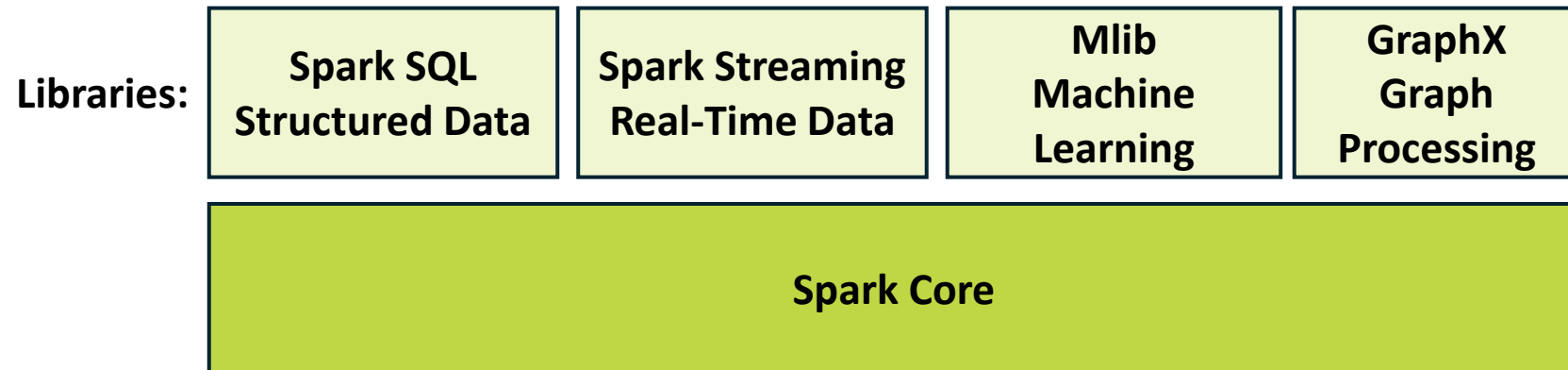
Ease of Use of Apache Spark

- Ease of Use: Offers high-level APIs in Java, Scala, Python, R, and SQL.
- The faster approach is when you use Scala as client language.
- But python is widely used due to the rich machine learning API and ease of use.



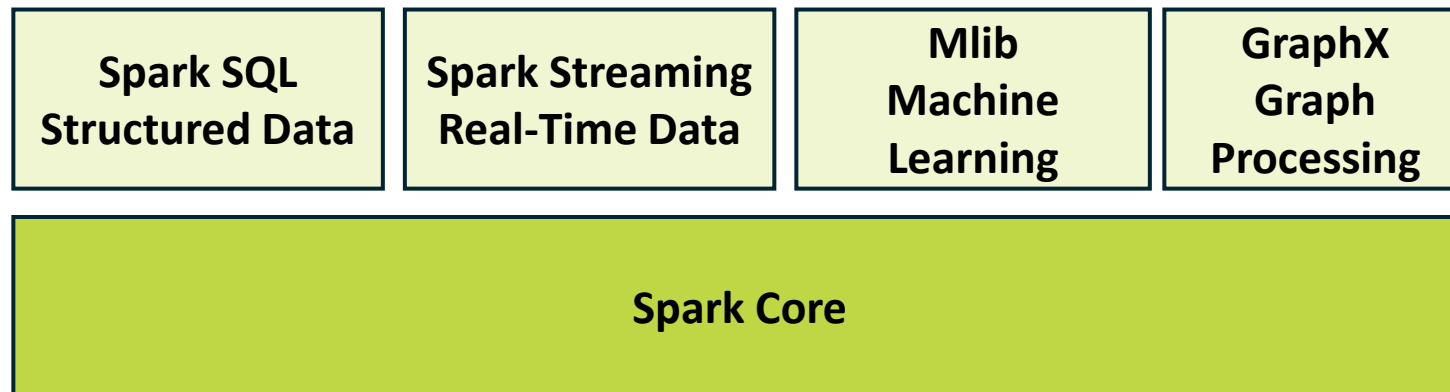
Unification of Apache Spark

- Unified Platform: Supports a wide range of workloads, including batch processing, streaming, machine learning, and graph processing.

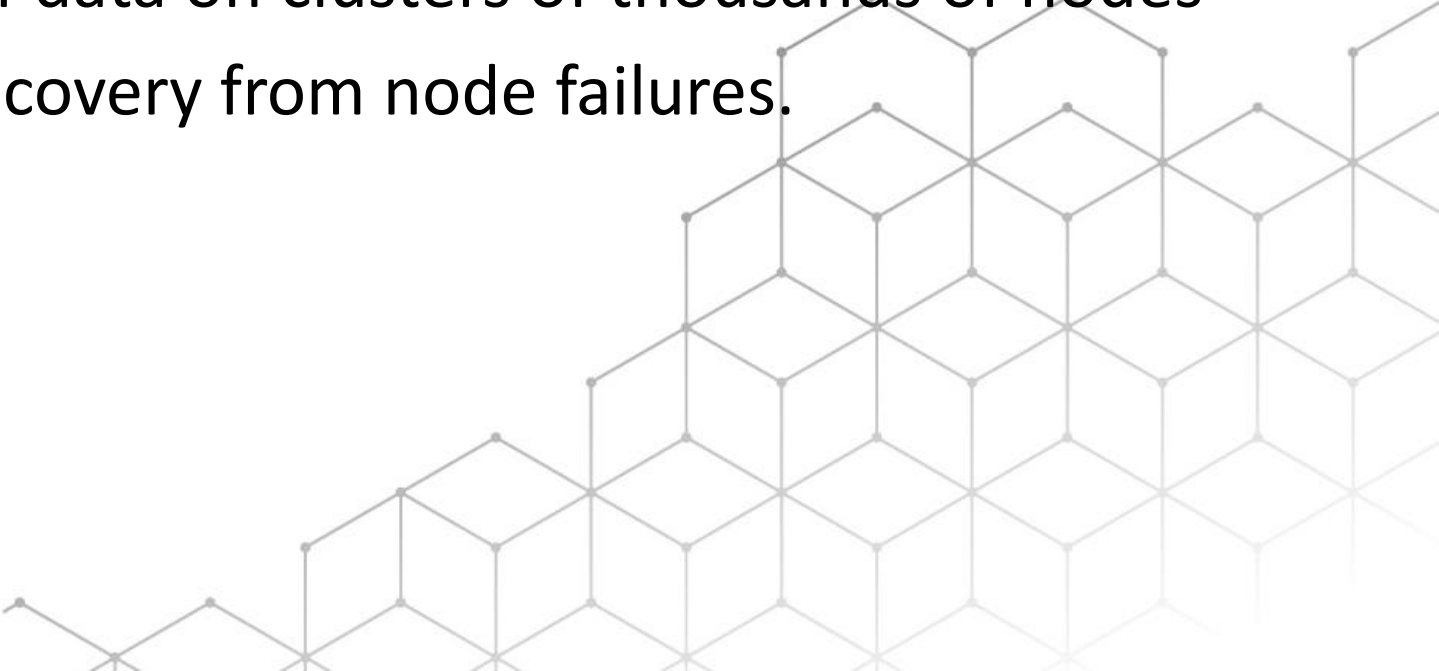


Use Cases

- Data warehousing
- Real-time data analytics
- Machine learning and AI pipelines
- Graph Computation
- Example 1: Processing logs from IoT devices using Spark Streaming.
- Example 2: Training a recommendation system using Spark MLlib.

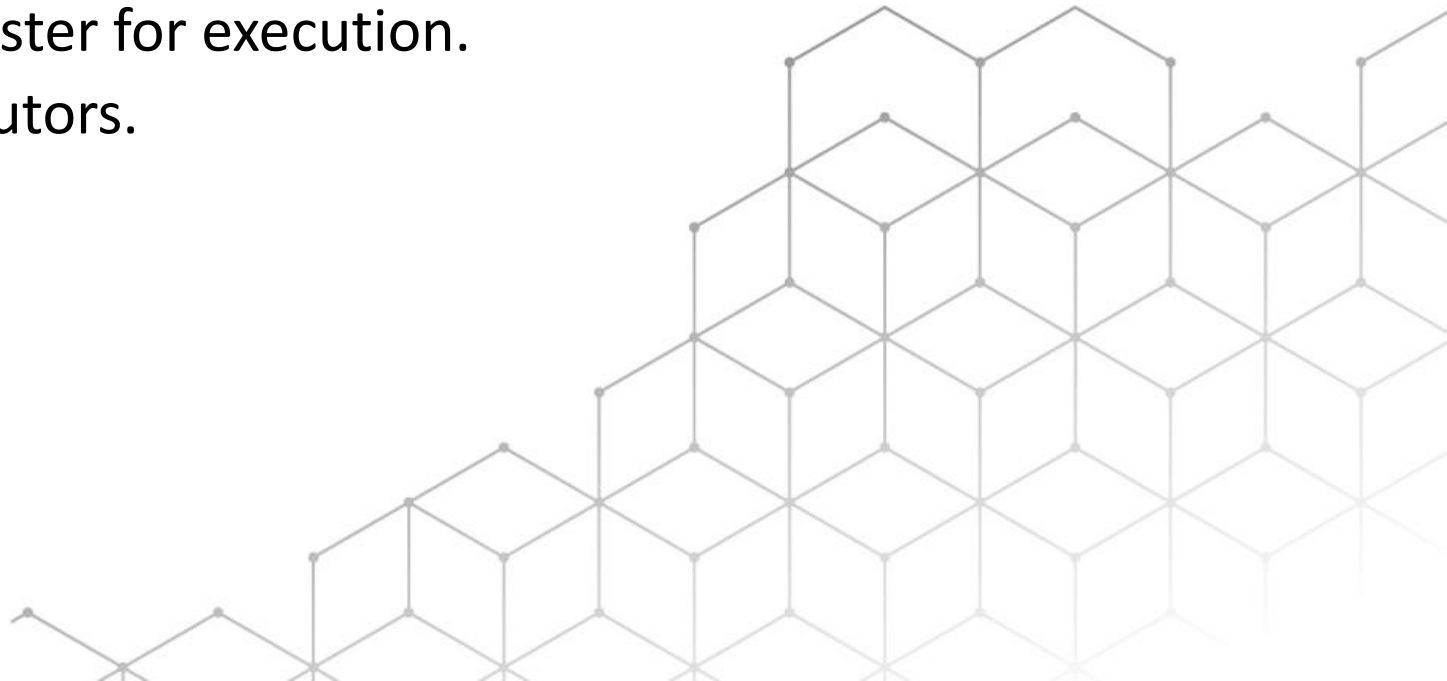


Advantages of Apache Spark

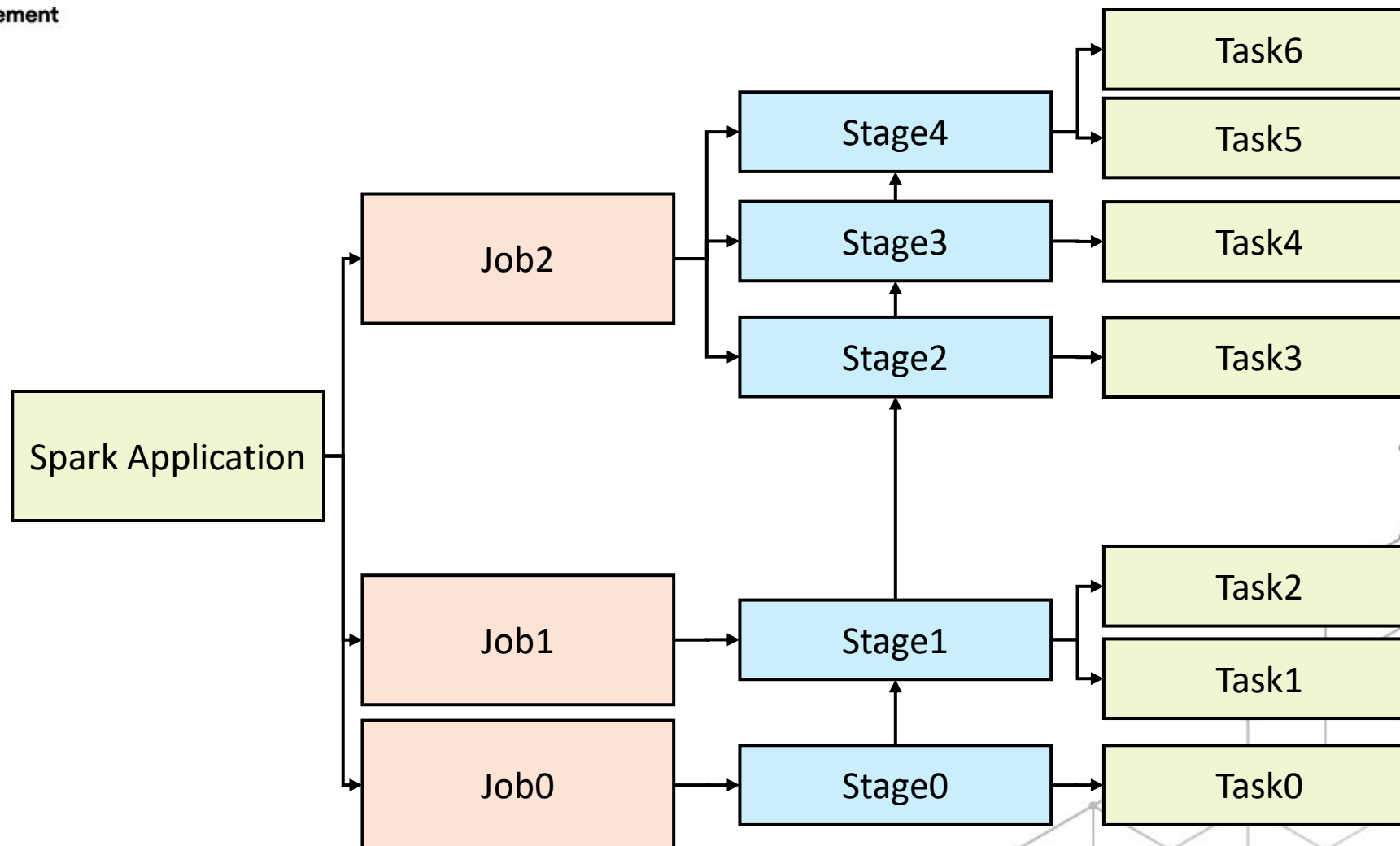
- **Performance**: In-memory data processing for faster computation
 - **Language Support**: Multi-language support: Python, Scala, Java, R
 - **Flexibility**: processes heterogeneous data (structured, unstructured)
 - **Scalability**: up to petabytes of data on clusters of thousands of nodes
 - **Fault Tolerance**: automatic recovery from node failures.
- 

Spark Application and Driver Program

- Definition: central part of Spark application that manages the execution of workflows.
- Key Responsibilities:
 - Converts user code into a Directed Acyclic Graph (DAG) of tasks.
 - Schedules tasks across the cluster for execution.
 - Collects results from the executors.



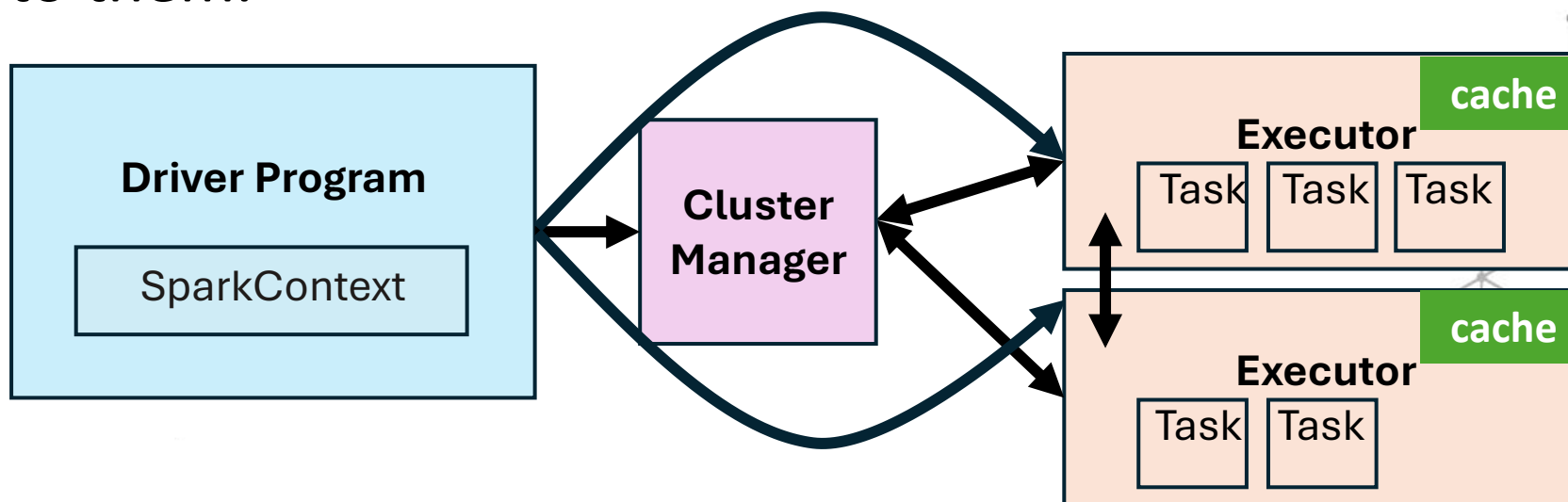
Execution Plan



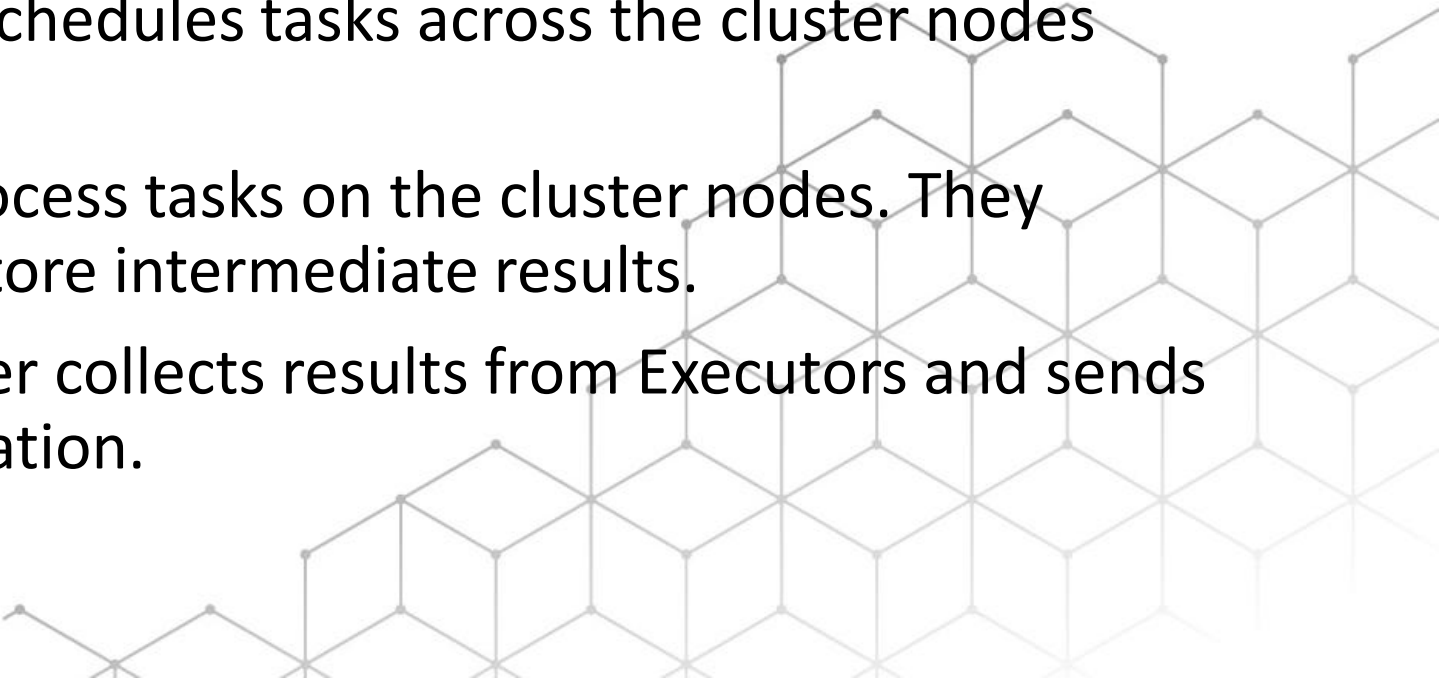
•Tasks are the smallest units of execution in Spark.

Core Spark Concepts

- At a high level, every Spark application consists of a **driver program** that launches various **parallel operations** on a cluster.
- The **driver program** contains your application's main function and defines distributed databases on the cluster, then applies operations to them.

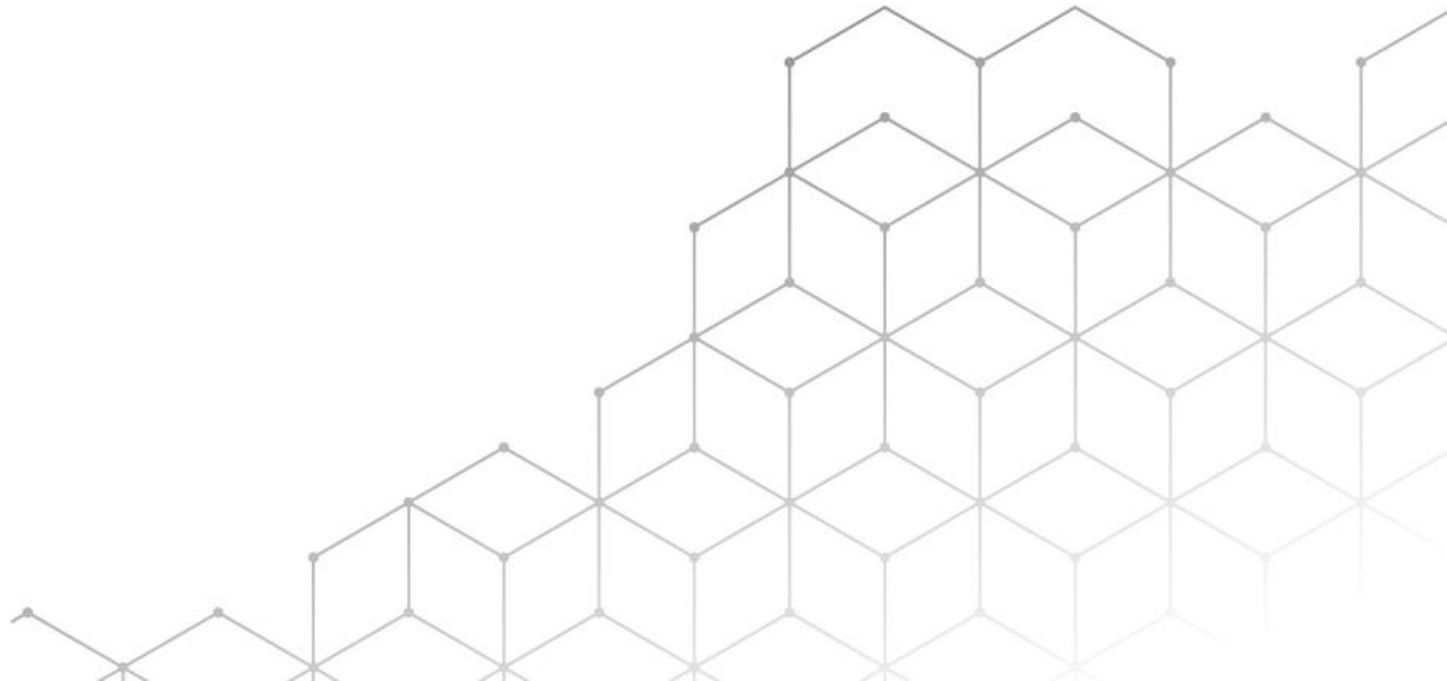


Workflow of Spark Application

1. **Job Submission**: The user submits an application (written in Python, Scala, Java, or R) to the Driver.
 2. **DAG Creation**: The Driver divides the application logic into smaller tasks, represented as a DAG.
 3. **Task Scheduling**: The Driver schedules tasks across the cluster nodes using the Cluster Manager.
 4. **Task Execution**: Executors process tasks on the cluster nodes. They perform computations and store intermediate results.
 5. **Result Aggregation**: The Driver collects results from Executors and sends them back to the user application.
- 

Disadvantages of Apache Spark

- Jobs failing with out-of-memory errors is very common and having many concurrent users makes resource management even more challenging



Next Class

- Distributed Processing and MapReduce
 - MLLib
 - Ecosystem tools



Questions and Thank you

